

Efficient GPU-Accelerated Adaptive Minimum Cost Seed Selection [Technical Report]

Gongyao Guo¹, Chen Feng¹, Yiran Li², Jieming Shi¹

¹The Hong Kong Polytechnic University, Hong Kong SAR, China ²University of Toronto, Toronto, Canada
gong-yao.guo@connect.polyu.hk, one.li@utoronto.ca, {chenfeng, jieming.shi}@polyu.edu.hk

Abstract

Efficient influence estimation and seed selection are crucial to social network advertising and are widely studied in data management. We focus on adaptive minimum cost seed selection (AMCSS), which selects *seed nodes* adaptively over multiple rounds, to reach a target number η of influenced users while minimizing total seed cost. Prior work, notably ASTI with multi-root reverse reachable sets (mRR-sets), remains computationally expensive on large graphs, often taking hours to days even with CPU parallelism.

We present GAAS, a GPU-accelerated method that integrates novel algorithmic and GPU-aware system designs to solve AMCSS in minutes. Processing mRR-sets involves irregular access patterns and variable-size samples, mismatching the GPU parallel architecture. Hence, we first develop a *GPU-tailored mRR-set structure*, GmRR, that assigns each thread block exclusive ownership of an equal-size *segment* with a *circular* layout, enabling efficient parallel mRR-set management while minimizing write contention. With GmRR, we design a GPU kernel `ParallelGen` to generate mRR-sets. Unlike prior work that regenerates mRR-sets from scratch in each round, we propose to update and reuse those from previous round, improving efficiency while requiring GPU-aware designs and rigorous theoretical analysis. Specifically, we design a `ParallelUpdate` kernel with *theoretically grounded update rules* that uses *circular segment updates* on GmRR for efficient mRR-set updates, together with a load-balancing scheme. We further devise a `Select` kernel for parallel seed selection. Integrating these together, GAAS efficiently solves AMCSS on GPUs with guarantees. Extensive experiments on large real-world graphs under different diffusion models show that GAAS is over an order of magnitude faster (up to 68.9×) than parallel CPU and GPU baselines, while the seed cost is among the lowest.

PVLDB Reference Format:

Gongyao Guo¹, Chen Feng¹, Yiran Li², Jieming Shi¹. Efficient GPU-Accelerated Adaptive Minimum Cost Seed Selection [Technical Report].

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/gongyao/GAAS>.

1 Introduction

Social networks are a dominant medium for information dissemination, where users share content and influence with each other. As of October 2025, over 5.6 billion people use social networks [10], driving a social advertising market exceeding \$136 billion [3].

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.

Advertisers select a set of influential users as *seeds*, each with a *cost*, to initiate diffusion process to influence a large number of users in the social network, to achieve a large *influence spread*. Identifying influential seeds requires accurate influence estimation, which is computationally expensive. This has motivated extensive work on scalable influence estimation and seed selection in data management [16, 17, 22–24, 26, 33, 34, 41, 42, 45, 47].

A typical formulation specifies a target number of influenced users η (*influence threshold*) and seeks a seed set of minimum cost whose influence spread reaches η , known as the minimum cost seed selection (MCSS) problem [12, 18, 20]. MCSS is non-adaptive: seeds are selected in a single batch without observing actual diffusion, so the realized spread can deviate substantially from η [12, 18].

Therefore, *adaptive* MCSS (AMCSS) has been proposed to select seeds iteratively over multiple rounds, where each round’s choices depend on observed diffusion outcomes [13, 14, 39, 44], *i.e., the nodes activated in previous rounds*. AMCSS is NP-hard, motivating efficient approximate algorithms with theoretical guarantees. ASTI [39] is state of the art that introduces multi-root reverse reachable sets (mRR-sets) to estimate user influence via stochastic breadth-first-search (BFS) from randomly selected root nodes. However, solving AMCSS on large graphs remains computationally prohibitive. ASTI often takes tens of thousands of seconds—more than one *day* for large η on large graphs—while a parallel CPU version of ASTI still needs around half a day. Such latency limits the practicality of AMCSS in time-critical applications.

We aim to improve the processing time to *minutes* (hundreds of seconds) while preserving result quality. This is challenging and existing methods largely ignore system design aspects. We achieve this by proposing a GPU-Accelerated AMCSS method, GAAS, which integrates novel algorithmic designs with GPU-aware optimizations to exploit the massive parallelism of modern GPUs.

GPUs are increasingly important for general-purpose parallel computing [21, 36, 38, 48–50]. Compared with CPUs, GPUs have a fundamentally different architecture. They execute massive numbers of lightweight threads in a SIMT (Single Instruction Multiple Thread) model, which suits data-parallel tasks with simple control flow and regular memory access but performs poorly under complex branching and irregular access patterns. The memory hierarchy also differs. Therefore, directly porting CPU-native algorithms to GPUs often results in suboptimal performance.

Specifically, the main cost in AMCSS is sampling and processing mRR-sets in each adaptive round for influence estimation. This involves irregular graph traversals and variable-size samples: the size of each mRR-set is unknown a priori and the required number of mRR-sets also changes by round, complicating GPU data structures

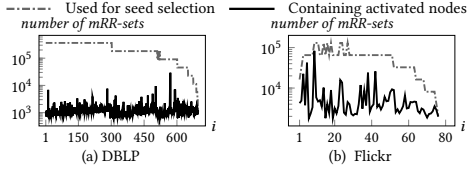


Figure 1: mRR-sets Statistics.

and parallelization. Moreover, existing methods regenerate all mRR-sets from scratch in every round, wasting computation despite the close relationship of mRR-sets between successive rounds.

Figure 1 shows the number of mRR-sets (log-scale) used in round i (dot-dash line) and the number of mRR-sets polluted by the activated nodes when round i finishes (solid line). The gap between the two lines indicates the mRR-sets that contain no activated nodes, i.e., not polluted. The gap is large in most rounds, e.g., in Figure 1(a) for DBLP. This suggests that many mRR-sets in round i have potential to be reused in round $i + 1$ after proper updating. It motivates GAAS to update and reuse mRR-sets. This is efficient but requires careful GPU-aware designs and theoretical analysis to ensure the updated mRR-sets are equivalent to newly generated ones.

To this end, we first design a *GPU-tailored mRR-set structure*, GmRR, that supports efficient storage, parallel generation, and update/reuse of mRR-sets across rounds in GAAS. GmRR partitions a pre-allocated global array into disjoint equal-size *segments*, each exclusively owned by one GPU thread block, minimizing write contention in parallel. Within each segment, a *circular* layout allows variable-length mRR-sets to be appended and updated with little data movement. All components in GmRR are flat arrays with regular access patterns, enabling efficient GPU execution. With GmRR, we design GPU kernels to parallelize mRR-set generation, update, and seed selection in GAAS. The ParallelGen kernel generates mRR-sets by evenly partitioning them across thread blocks and performing warp-centric parallel BFS within each block. We then develop the ParallelUpdate kernel to update mRR-sets from the previous round. We first derive *theoretically sound update rules* for reusing mRR-sets under changes in the graph and influence threshold, and then parallelize the update by letting each block update the mRR-sets in its own segment using a *circular segment update* on GmRR that enables in-segment updates and appends without excessive data movement, together with a load-balancing technique. With the mRR-sets ready, we devise a GPU kernel Select that selects each seed batch in parallel and updates data structures for next round. Integrating these components, GAAS solves AMCSS with an approximation ratio of $\ln \eta / [(1 - 1/e)(1 - \epsilon)]$, improving over existing study, and achieves high GPU efficiency.

Extensive experiments on real-world data under different diffusion models show that GAAS is typically over an order of magnitude faster than the baselines, while yielding seeds with comparable or lower total cost. Our contributions are summarized as follows.

- We design a GPU-accelerated method GAAS for solving AMCSS that significantly improves efficiency by integrating novel algorithmic designs with GPU-specific optimizations.
- We develop a GPU-tailored mRR-set structure GmRR that supports efficient storage, generation, and update of mRR-sets on GPUs, while reducing write contention and improving parallelism.

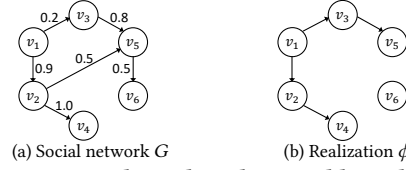


Figure 2: A social graph and a possible realization.

- We propose theoretically grounded mRR-set update rules across rounds, and GPU kernels with circular segment update on GmRR for efficient mRR-set update, generation, and seed selection.
- Extensive experiments show GAAS is often over an order of magnitude faster than baselines while maintaining low seed cost.

2 Preliminaries

2.1 Influence Diffusion Models

The social network is a directed graph $G = (V, E)$ with $|V| = n$ users and $|E| = m$ directed edges. Each edge $e = (u, v) \in E$ represents a social tie from u to v along which influence may diffuse. The in- and out-neighbor sets of u are denoted by $N_{\text{in}}(u)$ and $N_{\text{out}}(u)$. Each edge e has a *diffusion probability* $p(e) = p(u, v) \in [0, 1]$, and each node $u \in V$ has a positive *seed cost* $c(u)$, normalized to $(0, 1]$ by dividing all costs by the maximum. Figure 2(a) shows a graph G , where the number on each edge indicates its diffusion probability.

On graph G , given a *seed set* $S \subseteq V$, influence diffusion follows the widely adopted models, *independent cascade* (IC) or *linear threshold* (LT) [4, 39–41], in discrete steps. At step 0, only nodes in S are active, while all other nodes are inactive. When a node u becomes active at step t , it has one chance in step $t + 1$ to activate each inactive out-neighbor; afterwards, u makes no further attempts. The two models differ in how u activates an out-neighbor v . In the IC model, u activates v independently with probability $p(u, v)$. In the LT model, each node v has a threshold $\lambda_v \sim U(0, 1)$, and $p(u, v)$ is treated as a weight; v becomes active if the total weight from its active in-neighbors is no less than λ_v , i.e., $\sum_{u \in N_{\text{in}}(v)} \mathbb{I}_{\{u \text{ is active}\}} p(u, v) \geq \lambda_v$, where $\mathbb{I}_{\{\cdot\}}$ is the indicator function. The diffusion process stops when no new node is activated, and the total number of activated nodes is denoted by $I(S)$, which is the *influence spread* of S .

Note that $I(S)$ is a random variable due to the randomness in the diffusion process, which may lead to many possible outcomes. The IC propagation process can be equivalently described by the live-edge procedure [25]. Specifically, for each edge $e = (u, v) \in E$, we independently flip a coin with head probability $p(e)$ to decide whether e is *live* or *blocked*. All blocked edges are removed and the remaining graph is referred to as a *realization* of the probabilistic graph G , denoted by ϕ . Under a realization ϕ , the influence spread of a seed set S , denoted as $I_\phi(S)$, is the number of nodes reachable from S through live edges. For example, Figure 2(b) shows a realization ϕ with live edges marked while blocked edges removed in the IC model. Diffusing from seed v_2 over ϕ yields $I(\{v_2\}) = 2$, since v_2 can reach v_4 and itself. Similarly, for the LT model, a realization ϕ is obtained by sampling a threshold λ_v for each node v . The influence spread of S under ϕ is then the number of nodes that become active from S according to the sampled thresholds.

Let Ω be the set of all possible realizations and $p(\phi)$ denote the probability of a realization ϕ . For ϕ , the influence spread $I_\phi(S)$ is

deterministic. Thus, the expected spread is $\mathbb{E}_\Phi[I(S)] = \sum_{\phi \in \Omega} I_\phi(S) \cdot p_\phi$, which is the weighted average spread over all realizations in Ω .

2.2 Adaptive Minimum Cost Seed Selection

Non-adaptive MCSS identifies a seed set S with minimum cost such that $\mathbb{E}_\Phi[I(S)] \geq \eta$, without knowing the actual realization in advance [12, 18, 51]. As a result, the realized number of activated users may be far fewer than η in some realizations, or more than η in others. In contrast, the adaptive setting AMCSS selects seeds over multiple rounds, with each round's choices based on the observed diffusion outcomes so far. This feedback enables more cost-effective decisions and keeps the realized spread close to the target η , making AMCSS more practical in real deployments. Specifically, in round i , we select a batch of b seeds S_i on G_i , trigger diffusion from S_i , and observe the activated nodes A_i (including S_i). We then update the remaining influence threshold to $\eta_{i+1} = \eta_i - |A_i|$ and remove A_i and their incident edges to obtain the residual graph $G_{i+1} = (V_{i+1}, E_{i+1})$. Initially, $G_1 = G$, $V_1 = V$, $E_1 = E$, and $\eta_1 = \eta$. The process stops once at least η users are activated, and the final seed set is $S = \bigcup_i S_i$.

An adaptive policy π specifies how seeds are selected in each round. For a realization ϕ , let $S(\pi, \phi)$ be the seeds chosen by π , and let $C(\pi, \phi) = \sum_{v \in S(\pi, \phi)} c(v)$ be their total cost. The AMCSS problem is formally in Definition 2.1; it is NP-hard by reduction from adaptive seed minimization (ASM) with unit costs $c(v) = 1$ for all v , which is NP-hard [39].

Definition 2.1 (The AMCSS Problem). Given a graph $G = (V, E)$ with propagation probability $p(e)$ for each $e \in E$, cost $c(v)$ for each $v \in V$, and an influence threshold η , AMCSS seeks a policy π^* that minimizes the expected cost of selected seeds while ensuring the influence spread is at least η on any realization $\phi \in \Omega$. That is,

$$\pi^* = \operatorname{argmin}_\pi \mathbb{E}_\Phi[C(\pi, \Phi)] \quad \text{s.t. } |I(S(\pi, \phi))| \geq \eta, \forall \phi \in \Omega.$$

Since AMCSS only requires reaching η , we use the *truncated influence spread* $\Gamma_\phi(S) = \min\{I_\phi(S), \eta\}$ for each realization ϕ [39]. In round i , given previous seeds $S_{i-1} = \bigcup_{j=1}^{i-1} S_j$ that activated $I_\phi(S_{i-1})$ nodes, the *marginal truncated influence* of $S_i \subseteq V_i$ is $\Gamma_\phi(S_i | S_{i-1}) = \Gamma_\phi(S_i \cup S_{i-1}) - \Gamma_\phi(S_{i-1})$. Over all realizations, the expected versions are $\mathbb{E}[\Gamma(S | S_{i-1})] = \sum_\phi p_\phi \Gamma_\phi(S | S_{i-1})$ and $\mathbb{E}[\Gamma(S)] = \sum_\phi p_\phi \Gamma_\phi(S)$.

2.3 Graphics Processing Units

GPUs are widely used for general-purpose parallel computing [5, 28, 29, 36, 38]. The most popular GPU programming framework is Nvidia CUDA, which provides a C/C++-like programming interface.

When a GPU *kernel* is launched, the GPU creates a grid of threads organized in a hierarchical structure: the *grid* partitions threads into thread *blocks* of equal size (e.g., 1024 threads per block), and each block further divides into *warps*, containing 32 threads each. We employ one-dimensional grid with grid size and block size denoted by *gridDim.x* and *blockDim.x*, respectively. Grid size is the number of blocks, and block size is the number of threads in the block. GPUs follow the SIMT model, where all threads within a warp execute the same instruction in lockstep. This makes GPUs particularly efficient for data-parallel tasks with simple control flow, in contrast to CPU threads which can handle complex branching logic independently.

GPUs also feature a hierarchical memory architecture. The *global memory* is the largest memory accessible by all threads, while *shared memory* is a smaller memory shared among threads within the same

Algorithm 1: ASTI

Input: Graph G , target η , estimation error ϵ , batch size b , cost $c(\cdot)$.
Output: Seed set S .

```

1  $i \leftarrow 1, G_i \leftarrow G, \eta_i \leftarrow \eta, \rho_b \leftarrow 1 - (1 - 1/b)^b, S \leftarrow \emptyset;$ 
2 while  $\eta_i > 0$  do
3    $\delta \leftarrow 1/n_i; \hat{\epsilon} \leftarrow (\epsilon - \delta)(1 - \delta); S_i \leftarrow \emptyset;$ 
4   Estimate  $\theta$  and  $\theta_{\max}$ ;  $\theta_i \leftarrow \theta; H \leftarrow \lceil \log_2(\theta_{\max}/\theta) \rceil + 1;$ 
5    $a_1 \leftarrow \ln(3H/\delta) + \ln\binom{n_i}{b}; a_2 \leftarrow \ln(3H/\delta);$ 
6   while  $\theta_i < \theta_{\max}$  do
7     Generate additional mRR-sets until total reaches  $\theta_i$ ;
8     while  $|S_i| < b$  do
9        $u \leftarrow \operatorname{argmax}_{v \in V_i} \frac{\Lambda(S_i \cup \{v\}) - \Lambda(S_i)}{c(v)};$ 
10       $S_i \leftarrow S_i \cup \{u\};$ 
11       $\Lambda^l(S_i) \leftarrow (\sqrt{\Lambda(S_i)} + 2a_1/9 - \sqrt{a_1/2})^2 - a_1/18;$ 
12       $\Lambda^u(S_i^o) \leftarrow (\sqrt{\Lambda(S_i)}/\rho_b + a_2/2 + \sqrt{a_2/2})^2;$ 
13      if  $\Lambda^l(S_i) > \rho_b(1 - \hat{\epsilon})\Lambda^u(S_i^o)$  then break;
14       $\theta_i \leftarrow 2\theta_i;$ 
15    $S \leftarrow S \cup S_i;$ 
16   Diffuse from  $S_i$ , and obtain the activated nodes  $A_i$ ;
17    $G_{i+1} \leftarrow G_i \setminus A_i; \eta_{i+1} \leftarrow \eta_i - |A_i|; i \leftarrow i + 1;$ 

```

block. Dynamic memory allocation within kernels is supported through the device heap. However, device heap allocations incur substantial overhead and cause fragmented memory layouts. Since multiple threads may update the same memory location concurrently, GPU programs must use atomic operations judiciously.

Therefore, GPUs excel at processing simple data structures (e.g., arrays) with regular access patterns (e.g., sequential or strided), but perform poorly on complex data structures and irregular random accesses. Straightforward parallelization of CPU algorithms on GPUs often yields suboptimal performance. GPU-accelerated algorithms should be carefully redesigned to leverage simple data layouts and regular memory access patterns for efficient execution.

2.4 mRR-set and ASTI

Here we review the most related studies as preliminaries, leaving the other related work to Section 3. The work of [13, 14] is the first to study AMCSS, proposing a greedy policy that, in each round, selects the seed with the maximum influence-to-cost ratio and achieves an approximation ratio of $(\ln \eta + 1)^2$. However, this policy assumes exact knowledge of truncated influence, which is #P-hard to compute [7]. To address this, Tang et al. introduce the *multi-root Reverse Reachable* set (mRR-set) to efficiently estimate truncated influence and design the ASTI method [39] outlined below.

In round i , an mRR-set on residual graph G_i is generated as follows: (i) sample $k_i = n_i/\eta_i$ roots uniformly at random from V_i ; (ii) run a stochastic reverse BFS from these roots, where each edge (u, v) is traversed with probability $p(u, v)$; and (iii) take all visited nodes (including the roots) as the mRR-set R . Since k_i may be non-integer, we realize it in expectation by sampling $\lceil k_i \rceil$ roots with probability $k_i - \lfloor k_i \rfloor$ and $\lfloor k_i \rfloor$ roots otherwise. A node set S covers an mRR-set R if $S \cap R \neq \emptyset$, and its *coverage* $\Lambda(S)$ is the number of mRR-sets it covers. Given θ mRR-sets, we estimate $\mathbb{E}[\Gamma(S | S_{i-1})]$ by $\hat{\Gamma}(S | S_{i-1}) = \eta_i \Lambda(S)/\theta$ to approximate $\mathbb{E}[\Gamma(S | S_{i-1})]$ with bounded error [39]: $(1 - 1/e)\mathbb{E}[\Gamma(S | S_{i-1})] \leq \mathbb{E}[\hat{\Gamma}(S | S_{i-1})] \leq \mathbb{E}[\Gamma(S | S_{i-1})]$, where expectation $\mathbb{E}[\hat{\Gamma}(S | S_{i-1})]$ is over mRR-set randomness.

The original ASTI assumes uniform seed cost, i.e., $c(v) = 1$ for all $v \in V$ [39]. Algorithm 1 extends ASTI to non-uniform costs. It iteratively selects seeds and triggers diffusion until the influence

threshold η is reached (Line 3). In each round i , ASTI selects a batch of b seeds S_i that approximates the optimal batch S_i^o within a factor $\rho_b(1 - 1/e)(1 - \epsilon)$, where $\rho_b = (1 - 1/b)^b$ is initialized at Line 1 and ϵ is the estimation error parameter. Initially, ASTI generates a small number θ_i of mRR-sets. Using these sets, it greedily selects b seeds with the largest coverage-to-cost ratio (Lines 8–10), removing the mRR-sets they cover after each selection. It then computes a lower bound $\Lambda^l(S_i)$ on the coverage of S_i and an upper bound $\Lambda^u(S_i^o)$ on the optimal coverage (Lines 11–12). If $\Lambda^l(S_i)$ approximates $\Lambda^u(S_i^o)$ within the required ratio, S_i is accepted, added to S (Line 15), and influence diffusion is triggered (Line 16), updating the residual graph and influence target for the next round at Line 17. Otherwise, θ_i is doubled (Line 14), and the process from Lines 7–14 is repeated; if θ_i reaches the maximum $\theta_{\max}$, S_i is also accepted. Overall, ASTI achieves an approximation ratio of $(\ln \eta + 1)^2 / [(1 - 1/e)(1 - \epsilon)]$.

ASTI is a milestone for solving the AMCSS problem, but in each round it regenerates mRR-sets from scratch, incurring substantial overhead. Even with CPU parallelization, ASTI still needs thousands of seconds on real graphs. Our goal is to reduce this to hundreds or even tens of seconds by exploiting GPUs. As discussed in Section 2.3, GPUs have fundamentally different compute and memory architectures, so achieving this requires new algorithmic designs and GPU-specific optimizations tailored to AMCSS.

3 Related Work

Minimum Cost Seed Selection (MCSS). The MCSS problem aims to find a seed set that minimizes the cost to achieve η . The problem shares the submodularity and monotonicity properties with the classic Influence Maximization (IM) problem [30]. Existing approaches can be divided into two categories. The first category focuses on reaching η only in expectation [12, 15, 18, 30], where selected seeds can guarantee the coverage meets η . However, the actual influence spread of the selected seed often falls significantly short of η . Therefore, they are not feasible for AMCSS that requires the selected nodes must influence at least η nodes in every realization. The other category addresses this practical limitation by providing probabilistic guarantees [12, 51], ensuring that with a predefined probability p the influence achieved by the selected seeds reaches η in a random realization. Thus, we can set a high probability that the selected seeds meet the target influence in practice.

Adaptive Seed Minimization (ASM). The adaptive setting in IM and MCSS receives considerable attention [6, 9, 11, 13, 14, 16, 19, 22, 35, 39, 43, 44, 52]. In the adaptive setting, nodes are seeded in batches at different rounds, and after each batch, an actual diffusion process is allowed to unfold, with the outcomes observed before deciding the next batch of seeds. Golovin and Krause [13] pioneered the extension of the classic submodularity and monotonicity to the adaptive setting, thereby establishing an important theoretical foundation. However, their work did not provide a practical algorithm tailored for ASM. Tang et al. [39] first introduced truncated influence, recognizing that influence beyond a threshold η provides no additional value, and they devised mRR-sets to estimate marginal truncated influence targeted for ASM. However, ASTI assumes uniform costs for all nodes and overlooks the fact that the cost of nodes are heterogeneous in real world. Moreover, ASTI is

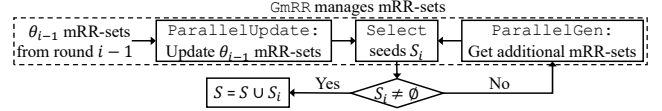


Figure 3: Overview of GAAS for round i .

a single-threaded CPU-based implementation and regenerates the entire collection of mRR-sets in each round.

Parallel RR-set Approaches. Several methods have been proposed to solve the IM problem in parallel [31, 32, 37] using the Reverse Reachable set (RR-set), which uses only one root. While the RR-set generation phase can be adapted for mRR-set generation in AMCSS, these approaches face notable efficiency challenges. Ripples and cuRipples [31, 32] utilize thread-level parallelization for RR-set generation. However, these methods suffer from workload imbalance due to skewed RR-set sizes and suboptimal thread utilization when RR-sets are fewer than the available threads. These issues remain when it is extended to mRR-set generation in adaptive rounds. For the block-level parallel approach [37], its implementation necessitates temporary storage and frequent atomic operations to manage RR-sets in global memory, also degrading performance.

4 GPU-Accelerated Method GAAS

4.1 Overview

GAAS is a GPU-centric framework for AMCSS, tightly coupling novel algorithmic designs with GPU-friendly mRR-set processing.

The dominant cost lies in sampling mRR-sets for influence estimation. On GPUs, efficiently managing these irregular, dynamically evolving samples is challenging. Further, to avoid regenerating mRR-sets from scratch in every round, we aim to reuse and update them across rounds, which introduces additional challenges in theoretical analysis, algorithm design, and GPU-parallel optimization.

Figure 3 outlines the GAAS method. After round $i - 1$, GAAS first invokes `ParallelUpdate` to update the θ_{i-1} mRR-sets from round $i - 1$ so they can be reused for round i , equivalent to generating new mRR-sets on the residual graph G_i . Then, `Select` attempts to select the seeds S_i , using these updated mRR-sets. If S_i is not empty, GAAS returns S_i for round i without generating new mRR-sets. Otherwise, GAAS invokes `ParallelGen` to get additional mRR-sets, and then invokes `Select` again until S_i is not empty. Throughout, mRR-sets are managed by the GmRR structure, which interacts with the GPU kernels for both updating and generating mRR-sets.

The GPU-tailored GmRR structure is for storing, managing, and updating mRR-sets (Section 4.2). GmRR reconciles the irregular, evolving nature of mRR-sets with massive GPU parallelism via a specialized global memory layout. Specifically, GmRR utilizes a pre-allocated global array partitioned into disjoint *segments* of equal size, each mapped exclusively to a specific thread block. This ownership enables blocks to generate and update mRR-sets locally, eliminating expensive dynamic memory allocation and excessive atomic contention. Internally, each segment functions as a *circular* buffer, allowing variable-length mRR-sets to be appended and updated efficiently within a segment, supporting the update of mRR-sets across rounds without frequent costly data shifting.

To efficiently generate mRR-sets on GPU, with GmRR, we propose the `ParallelGen` kernel (Section 4.3). At the inter-block level, it

evenly partitions mRR-set generation across thread blocks, which write mRR-sets into their own segments; at the intra-block level, it employs a GPU kernel Probe to perform stochastic BFS, where threads cooperatively traverse the nodes to construct an mRR-set.

To enable reuse of mRR-sets across rounds, we develop the GPU kernel ParallelUpdate with its supporting theory for mRR-set update (Section 4.4). An mRR-set from round $i-1$ reused in round i is *polluted* if it contains any node activated in round $i-1$, and *clean* otherwise. Both types must be carefully updated to preserve the theoretical properties of mRR-sets for accurate influence estimation: clean mRR-sets are augmented with additional roots to satisfy the sampling requirements on the residual graph G_i , whereas polluted mRR-sets are repaired by removing activated nodes and resampling roots together with their visited nodes. Building on these theoretical insights, we design the ParallelUpdate kernel over GmRR to implement mRR-set updates on GPUs efficiently. A key technique is *circular segment update* that treats each segment as a circular buffer, enabling efficient updates and appends within the segment, and eliminating expensive cumulative element shifts on the GPU. Moreover, we design a load-balancing scheme to mitigate inter-block workload skew from uneven numbers of polluted mRR-sets across segments, further improving the efficiency of ParallelUpdate.

Besides, we devise a GPU Select kernel (Section 4.5) for seed selection. It uses a parallel reduction to find the node with the maximum coverage-to-cost ratio, and marks all mRR-sets containing it as covered while decrementing the coverage of nodes in them. After selecting b seeds, Select evaluates confidence bounds to decide whether the batch is accepted or more mRR-sets are required.

4.2 GPU-Tailored mRR-set Structure GmRR

A naive idea to parallelize the algorithms is assigning each GPU thread the task of generating a full mRR-set, including root sampling and stochastic BFS. However, this results in highly irregular and heavy workloads per thread, leading to poor GPU utilization.

Another approach is block-level parallelization, where all threads in a block construct one mRR-set. However, this still incurs substantial overhead: the mRR-set size is unknown until completion, multiple blocks may contend for a global write pointer. Accordingly, the mRR-sets need to be buffered via device-heap allocations before being copied to global memory, which is inefficient on GPUs.

Furthermore, we aim to update and reuse existing mRR-sets from the previous round $i-1$ when selecting seeds in the current round i , improving overall efficiency. Hence, we design GmRR, a GPU-tailored structure for storing mRR-sets. At its core, GmRR uses a *segment*-based layout where each thread block is assigned a dedicated, non-overlapping segment in global memory to store its generated mRR-sets. Each block stores its mRR-sets contiguously in its own segment, and segments are mutually disjoint. This removes both the global write pointer (each block writes only to its segment) and temporary buffer allocations, since all elements are written directly to global memory. In addition, GmRR is implemented as flat *arrays*, enabling efficient access on the GPU. The per-block segments further enable dynamic updates of mRR-sets via a *circular layout within each segment*, which we will describe in more detail in Section 4.4.

Figure 4 shows a GmRR instance with two segments, each owned by one thread block. GmRR consists of three arrays: *mrr* stores all

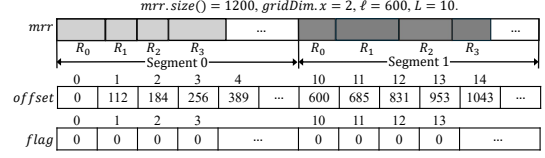


Figure 4: Example of GmRR with *mrr*, *offset* and *flag* arrays. mRR-sets, *offset* records start positions of mRR-sets in *mrr*, and *flag* marks whether an mRR-set is covered/polluted or not. The memory footprint of mRR-sets, i.e., the size of *mrr* is estimated as follows. In practice, given a new graph, we first get the theoretical upper bound on the number of mRR-sets needed in the first round, which is typically the largest across all rounds. We then generate a small sample of mRR-sets and measure their average size. Multiplying the two quantities gives the pre-allocated memory. This provides a practical safety margin without excessive overhead, so overflow rarely occurs. If it happens in a round, we simply allocate more memory and regenerate mRR-sets for that round.

Moreover, mRR-set memory usage across rounds is complex, maintaining a similar peak level but with increases and drops (see Figure 17 in the technical report [2]). Therefore, we do not resize the pre-allocated memory across rounds.

We partition *mrr* into $gridDim.x$ non-overlapping segments, where $gridDim.x$ is the number of launched thread blocks. Each segment has size ℓ that is the size of *mrr* divided by $gridDim.x$. For example, the size of *mrr* is 1200 and ℓ is 600 in Figure 4. Each segment j stores the mRR-sets generated by block j , starting at position $j \cdot \ell$ and ending at $(j+1) \cdot \ell - 1$ in *mrr*. The segment size ℓ is set by dividing the total size of *mrr* by $gridDim.x$. As explained in Section 2.4, in round i each mRR-set has k_i roots and a variable number of nodes reached by reverse BFS. We store it as a contiguous sequence of node IDs: the first k_i entries are the roots, followed by the visited nodes. Each block uses all its threads to generate mRR-sets and stores them contiguously within its segment. In Figure 4, Segment 0 (gray) stores the four mRR-sets R_0 – R_3 generated by Block 0, and Segment 1 (dark) stores those generated by Block 1.

In GmRR, mRR-sets in a segment are always stored contiguously. Thus, each mRR-set can be identified by the start positions of two contiguous mRR-sets in the *offset* array. For example, in Segment 0 the start positions of R_2 and R_3 are 184 and 256 at *offset*[2] and *offset*[3]; the nodes in R_2 occupy from *mrr*[184] to *mrr*[255]. The last mRR-set has its end position stored in *offset*[$r+1$], where r is the number of mRR-sets in the segment. For example, the end position of R_3 in Segment 0 is at *offset*[4], which is also the start position of free space in the segment for future mRR-sets.

Given the required number of mRR-sets per round, we evenly assign them to segments, simplifying in-segment lookup and balancing workload across segments and blocks. The remaining question is how many offset values to store in *offset* for each segment. The number of mRR-sets per segment may vary across rounds, but we do not want to reconstruct *offset* when this number changes. Since the global maximum number of mRR-sets over all rounds has an upper bound and the number of segments is fixed, we predefine a sufficiently large per-segment capacity L by dividing the upper bound by $gridDim.x$ and rounding the result up to the nearest power of two, and reserve L offset entries per segment. The size of the *offset* array is then $L \cdot gridDim.x$, since each segment corresponds one-to-one with a thread block. Every L entries in *offset*

record the start positions of at most L mRR-sets in the corresponding segment. The start position of the r -th mRR-set in the j -th segment is given by $offset[r + j \cdot L]$. For example, in Figure 4, L is set to 10, and thus the start position of the mRR-set R_1 in Segment 1 is $offset[1 + 1 \cdot L] = offset[11]$.

Accordingly, the *flag* array also has $L \cdot gridDim.x$ entries; entry $(r + j \cdot L)$ marks whether the r -th mRR-set in segment j is covered/polluted or not. Section 4.5 details how *flag* is updated.

4.3 Parallel mRR-set Generation

In round i , GAAS reuses and updates the θ_{i-1} mRR-sets from round $i - 1$. (Section 4.4). If the required number θ_i exceeds θ_{i-1} , we generate $\theta_i - \theta_{i-1}$ additional mRR-sets. Especially, in round 1, we generate θ_1 mRR-sets from scratch, by applying the GPU kernel *ParallelGen*, where each block uses all its threads to construct new mRR-sets and write them into its own segment in GmRR.

The ParallelGen Kernel. Algorithm 2 presents the *ParallelGen* kernel for mRR-sets generation in a block *block.id*. In GPUs, block IDs are assigned contiguously from 0 to $gridDim.x - 1$, which we also use to index the segments in GmRR. As mentioned, when θ mRR-sets are needed, we let each segment store on average $\bar{\theta} = \theta / gridDim.x$ mRR-sets. Often, $\bar{\theta}$ is not an integer. In this case, we require the first $\bar{\theta} \cdot gridDim.x$ segments to store $\lceil \bar{\theta} \rceil$ mRR-sets, and the remaining segments to store $\lfloor \bar{\theta} \rfloor$ mRR-sets. Therefore, in Line 1 we compute the number of existing mRR-sets r in the segment owned by block *block.id* as $\theta_{i-1} / gridDim.x$, plus one if $block.id < \theta_{i-1} \% gridDim.x$. Likewise, in round i with θ_i mRR-sets required, we set the target number r' of mRR-sets in this segment to $\theta_i / gridDim.x + \mathbb{I}_{\{\theta_i \% gridDim.x > block.id\}}$. The difference $r' - r$ is the number of new mRR-sets that block *block.id* generates in round i and stores in its segment. In Line 3, we set the starting write position b_w for the next mRR-set as the end of the last existing mRR-set in the segment, i.e., $offset[r + block.id \cdot L]$. We also initialize $b_r \leftarrow b_w$, where b_r is the read pointer used by *Probe* (Algorithm 3) for stochastic BFS within the current mRR-set.

Each iteration of the while loop (Lines 4–12) constructs one mRR-set using all threads in the block. We first mark all nodes in the residual graph G_i as unvisited in parallel (Line 5). Next, we uniformly and parallelly sample k_i distinct roots from V_i (Line 7): thread j samples a root u and writes it to *mrr* at position $(b_w + j) \cdot \ell + block.id \cdot \ell$ (Line 8). This modulo-based address within the segment enables circular updates later. Line 9 atomically increments $cnt[u]$, the *coverage count* of node u (the number of mRR-sets containing u). We say that u covers an mRR-set if it appears in that set. After all k_i roots are written, *Probe* (Line 11) performs reverse BFS from these roots, appending newly visited nodes to the mRR-set and updating b_w , *mrr*, and *cnt*. Finally, we advance r , record the end position of the new mRR-set in $offset[r + block.id \cdot L] \leftarrow b_w$, and set $b_r \leftarrow b_w$ to start the next mRR-set generation (Line 12).

The Probe Kernel. Given arbitrary k roots, *Probe* performs reverse BFS from the roots to complete the mRR-set. b_r denotes the start of the current mRR-set and is used to index frontier nodes whose in-neighbors are probed. b_w denotes the next write position in *mrr* within the segment of *block.id*. We maintain a BFS queue indexed by *head* and *tail*, initialized to 0 and k respectively, so that $tail - head$ equals the number of frontier nodes pending expansion.

Algorithm 2: The ParallelGen Kernel

Input: Graph G_i , coverage counter *cnt*, *mrr*, *offset*, θ_{i-1} , θ_i .

```

1  $r \leftarrow \theta_{i-1} / gridDim.x + \mathbb{I}_{\{\theta_{i-1} \% gridDim.x > block.id\}}$ ;
2  $r' \leftarrow \theta_i / gridDim.x + \mathbb{I}_{\{\theta_i \% gridDim.x > block.id\}}$ ;
3  $b\_w \leftarrow offset[r + block.id \cdot L]$ ;  $b\_r \leftarrow b\_w$ ;
4 while  $r < r'$  do
5    $\forall u \in V_i, visited[u] \leftarrow false$  in parallel;
6   parallel for  $j = thread.id; j < k_i; j += blockDim.x$  do
7     Atomically sample  $u \in V_i$  with  $visited[u] == false$  and set
        $visited[u] \leftarrow true$ ;
8      $mrr[(b\_w + j) \% \ell + block.id \cdot \ell] \leftarrow u$ ;
9     atomicAdd( $cnt[u], 1$ );
10   $__syncthreads()$ ;
11  Probe( $G_i, k_i, b\_r, b\_w, mrr, visited, cnt$ );
12   $r \leftarrow r + 1$ ;  $offset[r + block.id \cdot L] \leftarrow b\_w$ ;  $b\_r \leftarrow b\_w$ ;

```

Each warp cooperatively processes the in-neighbors of a single frontier node, while different warps handle different frontier nodes. The lane identifier *lane.id* of each thread is *thread.id* mod 32. The loop in Line 2 proceeds while the BFS queue is non-empty. In each iteration, the first thread in the warp (*lane.id* = 0) obtains the frontier index *idx* (Line 3), which is then broadcast to all threads in the warp (Line 4). If *idx* is valid, i.e., smaller than *tail* (Line 5), each warp retrieves the frontier node u (Line 6). The threads in warp iterate over a u 's in-neighbors v (Line 8), and check whether edge (v, u) satisfies the IC or LT activation condition in Section 2.1 (Line 9). If so, unvisited v is appended to the mRR-set at the next circular position in *mrr* by atomically incrementing b_w (Line 11), and we update $visited[v]$, $cnt[v]$, and the BFS queue *tail* (Line 12).

4.4 Parallel mRR-set Update

In round i , we reuse mRR-sets from round $i - 1$. After selecting seeds S_{i-1} and simulating diffusion, we obtain activated nodes A_{i-1} and form the residual graph G_i by removing A_{i-1} and their incident edges from G_{i-1} . An mRR-set from round $i - 1$ is *polluted* if it contains any node in A_{i-1} ; otherwise it is *clean*.

Such reuse is not straightforward, but requires proper updates to both clean and polluted mRR-sets, as analyzed in Lemma 4.1 and 4.3 later, so that each updated mRR-set has exactly the same distribution as an mRR-set generated directly on G_i , to preserve the theoretical properties needed for accurate influence estimation. In round i , each mRR-set must contain $k_i = n_i / \eta_i = (n_{i-1} - |A_{i-1}|) / (\eta_{i-1} - |A_{i-1}|)$ roots, and k_i is monotonically non-decreasing over rounds (i.e., $k_i \geq k_{i-1}$), since $|A_{i-1}| \geq 1$ and $n_i \geq \eta_i$. Hence, even clean mRR-sets must be updated to satisfy the new root requirement. Moreover, updating mRR-sets in GmRR requires dedicated designs to avoid excessive data movement on the GPU.

Below, we first analyze updates of clean and polluted mRR-sets, then present a *circular segment update* strategy over GmRR, and finally introduce the *ParallelUpdate* kernel.

Given the activated node set A_{i-1} from round $i - 1$, we first identify polluted mRR-sets in GmRR by invoking the *Discover* kernel (Algorithm 6) with A_{i-1} as input: every mRR-set containing a node in A_{i-1} is marked as polluted by setting its entry in the *flag* array to true, and the coverage counts of nodes in these polluted mRR-sets are decremented to remove their contribution in round i . The *flag*

Algorithm 3: The Probe Procedure

Input: Graph G_i , root number k , b_r , b_w , mrr , $visited$, cnt

```

1  $b_w \leftarrow b_r + k$ ;  $head \leftarrow 0$ ;  $tail \leftarrow k$ ;  $lane.id \leftarrow thread.id \% 32$ ;
2 while  $head < tail$  do
3   if  $lane.id == 0$  then  $idx \leftarrow atomicAdd(head, 1)$ ;  $\_\_syncwarp(0)$ ;
4   Broadcast  $idx$  to threads in the same warp;
5   if  $idx < tail$  then
6      $u \leftarrow mrr[(b_r + idx) \% \ell + block.id \cdot \ell]$ ;
7     parallel for  $j = lane.id; j < |N_{in}(u)|; j += 32$  do
8       Get the  $j$ -th node  $v \in N_{in}(u)$ ;
9       if  $p(v, u)$  meets the condition in IC or LT then
10        if  $visited[v] == false$  and  $set\ visited[v] \leftarrow true$  then
11           $mrr[atomicAdd(b_w, 1) \% \ell + block.id \cdot \ell] \leftarrow v$ ;
12           $atomicAdd(cnt[v], 1)$ ;  $atomicAdd(tail, 1)$ ;
13    $\_\_syncwarp(0)$ ;
```

array is reused in Section 4.5 for seed selection. Thus, we defer the details of Discover to there and focus here on updating mRR-sets.

Theoretical Analysis on Updating a Clean mRR-set. Let R be a clean mRR-set from round $i - 1$ with root set K of size k_{i-1} . Because R contains no node in A_{i-1} , its reverse BFS and root distribution remain valid on G_i . Since $k_i \geq k_{i-1}$, we update R by keeping all existing nodes and appending $k_i - k_{i-1}$ additional roots and their reverse BFS expansions to meet the root requirement in round i .

Specifically, in Lemma 4.1, we show that when adding $k_i - k_{i-1}$ new roots, instead of sampling them all from the residual graph G_i , we can split the new roots into two subsets. First, we can sample k_∇ nodes uniformly from $R \setminus K$ and promote them to roots. Since these promoted roots already exist in R , they do not require additional reverse BFS, saving computation. The remaining $k_\Delta = k_i - k_{i-1} - k_\nabla$ new roots are sampled uniformly at random from $V_i \setminus R$ in G_i and expanded via reverse BFS by the Probe kernel. Lemma 4.1 establishes the theoretical correctness of this update strategy. We provide proof sketches here and all proofs are in technical report [2].

LEMMA 4.1. *If the previous mRR-set R with k_{i-1} roots K is clean, the nodes in R can be preserved. Additionally, we can select $k_\Delta = (k_i - k_{i-1}) \cdot \frac{n_i - |R|}{n_i - k_{i-1}}$ roots from $V_i \setminus R$ and the rest $k_\nabla = k_i - k_{i-1} - k_\Delta$ roots from $R \setminus K$ uniformly at random. The resultant root set has the same distribution as one directly generated from V_i .*

PROOF SKETCH. To prove that R can be preserved, we show that the distribution of nodes already in R matches that in a newly generated mRR-set. Our root-addition strategy is correct because, even if a new root coincides with a node already in R , the content of R remains unchanged, and the additional roots selected from V_i follow the same distribution as those in a fresh mRR-set. $\square$

Theoretical Analysis on Updating a Polluted mRR-set. For a polluted mRR-set R from round $i - 1$, since it contains at least one node in A_{i-1} , its reverse BFS generated on G_{i-1} is no longer valid on G_i . A naive remedy is to discard all polluted mRR-sets and regenerate them by sampling fresh roots from G_i . However, this alters the global distribution of root sets over all mRR-sets and thus breaks the approximation guarantee, as illustrated in Example 4.2.

Example 4.2. Assume the algorithm starts with $\eta = 4$ on graph G under the IC model in Figure 2(a). We sample sufficiently many mRR-sets, each with $\lceil n/\eta \rceil = 2$ roots. Figure 5(b) enumerates all $\binom{6}{2} = 15$ possible root sets, each with probability $1/15$. In the first round, v_2 is selected and, under the realization in Figure 2(b), activates v_2 and v_4 . In the next round, we work on the residual graph G_2 in

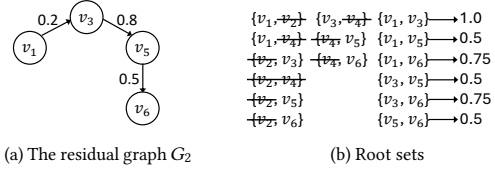


Figure 5: A residual graph and root sets (activated nodes are struck through).

Figure 5(a) with $\eta_2 = 2$, still requiring mRR-sets with $\lceil n_2/\eta_2 \rceil = 2$ roots. If, during update, we regenerate every polluted mRR-set by resampling both roots uniformly from the remaining nodes, the root distribution becomes biased. For instance, $\frac{3}{5}$ of the mRR-sets whose root set contains v_2 or v_4 are regenerated, while mRR-sets such as $\{v_1, v_3\}$ never include v_2 . In total, $\frac{11}{15}$ mRR-sets are regenerated, and the final probability of sampling root set $\{v_1, v_3\}$ becomes $\frac{1}{15} + \frac{11}{15} \cdot \frac{1}{6} = \frac{17}{90}$, deviating from the uniform probability $\frac{1}{6}$ and thus introducing sampling bias.

To address this, Lemma 4.3 shows that we should retain all non-activated roots in polluted mRR-sets and resample only the activated ones. For a polluted mRR-set R with root set K of size k_{i-1} , let $K_1 = K \setminus A_{i-1}$ and $K_2 = K \cap A_{i-1}$. We discard the roots in K_2 and sample $|K_2| + k_i - k_{i-1}$ new roots uniformly at random from $V_i \setminus K_1$; by Lemma 4.3, the resulting root set has the same distribution as one generated directly from V_i . We then invoke the Probe kernel to perform reverse BFS from these roots to update the mRR-set.

LEMMA 4.3. *Let K be the set of roots of a random mRR-set R from round $i - 1$. We write $K = K_1 \cup K_2$, where K_1 is not activated while K_2 is. Then, in round i , we delete K_2 from K and select $|K_2| + k_i - k_{i-1}$ nodes from $V_i \setminus K_1$ uniformly at random. The distribution of the new root set is the same as the one generated from V_i directly.*

PROOF SKETCH. Let K' be a set of k_i nodes sampled at random from V_i , and let Z be any k_i -subset of V_i . For a freshly generated mRR-set on G_i , $\Pr[K' = Z] = 1/C_{n_i}^{k_i}$. We then show that our method also selects Z with probability $1/C_{n_i}^{k_i}$. Hence, it induces the same root-set distribution as fresh generation. $\square$

Example 4.4. To illustrate Lemma 4.3, we revisit Example 4.2. We keep all clean mRR-sets and retain non-activated roots in polluted ones. For root set $\{v_1, v_3\}$, $\frac{2}{15}$ of mRR-sets already contain v_1 or v_3 . The resampling of the other root results in $\{v_1, v_3\}$ with probability $\frac{1}{3}$, contributing $\frac{2}{15} \cdot \frac{1}{3}$. Among the $\frac{11}{15}$ mRR-sets with roots $\{v_2, v_4\}$, regeneration yields $\{v_1, v_3\}$ with probability $\frac{1}{6}$, contributing $\frac{11}{15} \cdot \frac{1}{6}$. The original $\frac{1}{15}$ mRR-sets with roots $\{v_1, v_3\}$ remain unchanged. Thus the final probability of $\{v_1, v_3\}$ is $\frac{2}{15} \cdot \frac{1}{3} + \frac{11}{15} \cdot \frac{1}{6} + \frac{1}{15} = \frac{1}{6}$. A symmetric argument shows every root set has probability $\frac{1}{6}$, confirming unbiased root-set sampling.

Circular Segment Update of mRR-sets on GmRR. With the theoretical analysis, we now describe how to efficiently update mRR-sets within a GmRR segment. Recall that each thread block exclusively owns one segment and is responsible for generating and updating its mRR-sets. The top-left part of Figure 6 illustrates Segment 0 with four mRR-sets $R_{0,1,2,3}$ generated by Block 0 in round $i - 1$, where R_0 and R_3 are clean, while R_1 and R_2 are polluted as indicated in the flag array. In round i , all four mRR-sets must be updated.

A straightforward approach is to update mRR-sets sequentially from R_0 to R_3 . Since R_0 is clean, we add new roots and perform

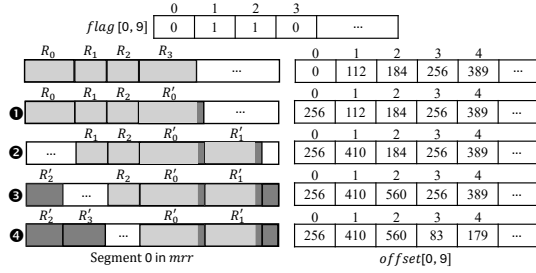


Figure 6: Circular segment update of a segment in GmRR.

reverse BFS, increasing its size and forcing us to shift subsequent mRR-sets R_1 – R_3 backward to free space for the expanded R_0 . Updating the polluted mRR-set R_1 likewise changes its size after root resampling and reverse BFS, so all subsequent mRR-sets must be shifted again. Repeating this process for all mRR-sets requires moving each set multiple times, which is inefficient when many mRR-sets reside in a segment, and the unknown final size of each updated mRR-set also requires temporary GPU buffers.

Instead, we design a circular segment update strategy that moves each mRR-set at most once, and writes new content circularly within the segment, so no subsequent mRR-sets are shifted. Steps ①–④ in Figure 6, together with the *offset* array, illustrate this process. As explained in Section 4.2, each GmRR segment stores mRR-sets sequentially in *mrr*, with *offset* recording the start of their positions and the free space. Especially, we first update the last mRR-set in the segment in place. In Figure 6, the last mRR-set R_3 is updated to R'_0 , which becomes the first mRR-set in the segment for round i (Step ①); its light-gray region is the reused content, and its dark-gray region is the part newly probed from added roots. Accordingly, we set *offset*[0] = 256, the start position of R'_0 , which also indicates that R'_0 is now the first mRR-set in the segment.

Next, we update R_0 from round $i-1$ to R'_1 . Instead of modifying R_0 in place (which would require shifting subsequent mRR-sets), we write R'_1 immediately after R'_0 , leaving the original space of R_0 free (white in Step ②). Originally, R_0 is the first mRR-set; after the update it becomes the second, at *offset*[1] = 410.

We then update R_1 to R'_2 and place it after R'_1 , where R'_2 has start position *offset*[2] = 560, making it the third mRR-set in the segment (Step ③). When the segment end is reached, the remaining free space lies at its beginning. Using circular addressing within the segment (mapping each physical index modulo the segment length ℓ), the portion of R'_2 that exceeds the segment end is wrapped to the segment front (dark gray in Step ③). The free space after R'_2 is then used to update R_2 into R'_3 with start position *offset*[3] = 83; since it is the last mRR-set, its end position is stored at *offset*[4] = 179, which also indicates the start of the free space (Step ④).

The ParallelUpdate Kernel. Algorithm 4 presents the kernel *ParallelUpdate*, which updates mRR-sets in each segment by a thread block. The input is the θ_{i-1} mRR-sets in GmRR from round $i-1$, represented by *mrr* and *offset*, and the array *flag* marking each mRR-set as polluted or clean. For each block, we first compute its local mRR-set count r (Line 1), set r_1 to index the updated mRR-sets and r_2 to index the mRR-set being processed (Line 2), where $r_2 = r - 1$ so that we start update from the last mRR-set. ℓ_b is the base address of the block’s segment. We then read the *start* and *end* of the r_2 -th mRR-set and set b_w to *start* as the write position for

its updated version (Lines 3–4). In each iteration (Line 5), we mark all nodes in V_i as unvisited in parallel, then set both b_r and b_w to the new start position of the mRR-set pending update (Line 6), and compute its original length *len* under the circular layout (Line 7).

From Lines 8 to 18, if the r_2 -th mRR-set is clean, we first obtain k_Δ and k_∇ according to Lemma 4.1 (Line 9). We then retain all original nodes of this mRR-set by copying them to their new position and marking them as visited in the updated mRR-set (Lines 10–12), with all operations executed in parallel. Next, we sample k_Δ new roots and write each new root to position $(b_w + \text{len} + j) \% \ell + \ell_b$ of *mrr*, updating the *visited* and *cnt* arrays accordingly (Lines 13–15). We then perform reverse BFS from these k_Δ new roots by invoking the *Probe* kernel (Line 17). Note that the k_Δ new roots and k_∇ promoted roots are not necessarily contiguous with the original roots of this mRR-set. As described in Section 4.2, each mRR-set in *mrr* stores all roots first, followed by nodes probed by reverse BFS. Thus, in Line 18 we swap the positions of the k_Δ and k_∇ roots so that the first k_i entries of the updated mRR-set are its roots.

From Lines 20 to 26, if the r_2 -th mRR-set is polluted, we first retrieve its previous roots at their previous positions in the existing mRR-set (Lines 21–22) and, for each root u , test whether $u \in V_i$. If $u \notin V_i$ (i.e., it was activated in the previous round) or fewer than k_i valid roots remain, we sample replacement roots so that the updated mRR-set still has exactly k_i roots (Lines 24–25). Finally, we write all roots to their new positions in *mrr* and atomically increase the coverage of each root by 1 (Line 26). Then *Probe* is invoked from these k_i roots to get the mRR-set (Line 28). Once the r_2 -th mRR-set is updated, we move to the next one by updating r_2 , recomputing its original *start* and *end*, recording the start position of the updated mRR-set in *offset*, and incrementing r_1 (Lines 29–31). After all mRR-sets are processed, we set b_w as end of last updated mRR-set, which is also the start of the free space in the segment (Line 32).

Update Workload Balancing. In *ParallelUpdate*, the main cost comes from updating polluted mRR-sets, which requires more BFS traversals than clean ones. Thus, the workload per segment in GmRR is largely determined by its number of polluted mRR-sets. This can be highly skewed across segments, causing severe inter-block imbalance. To alleviate this, we adopt a lightweight load-balancing scheme. Using the *flag* array, we first count in parallel the polluted mRR-sets in each segment, sort all segment IDs in decreasing order of this count and partition them into β groups, each containing as many segments as blocks. Segments in the same group have similar workloads, and *ParallelUpdate* processes groups one by one over β iterations, which reduces workload imbalance among blocks.

The time breakdown in Figure 12 of Section 5.4 shows that, in GAAS, the *ParallelUpdate* kernel significantly reduces the time spent on mRR-set generation compared with a regeneration-based variant GAAS_{gen} with its time dominated by mRR-set generation. Consequently, the overall runtime of GAAS is much lower, demonstrating the substantial benefit of reusing and updating mRR-sets.

4.5 Parallel Seed Selection

In round i with θ_i mRR-sets, we greedily select a batch S_i of b seeds that maximizes the coverage-to-cost ratio. We use the GPU kernel *Select* (Algorithm 5) to perform one seed-selection attempt. We initialize $S_i = \emptyset$ with $\Lambda(S_i) = 0$, copy *cnt* to the working array

Algorithm 4: The ParallelUpdate Kernel

Input: Graph G_i , cnt , mrr , $offset$, $flag$, θ_{i-1} .

```

1  $r \leftarrow \theta_{i-1} / \text{gridDim.x} + \mathbb{I}_{\{\theta_{i-1} \% \text{gridDim.x} > \text{block.id}\}}$ ;
2  $r_1 \leftarrow 0$ ;  $r_2 \leftarrow r - 1$ ;  $\ell_b \leftarrow \text{block.id} \cdot \ell$ ;
3  $start \leftarrow offset[r_2 + \text{block.id} \cdot L]$ ;  $b\_w \leftarrow start$ ;
4  $end \leftarrow offset[r_2 + 1 + \text{block.id} \cdot L]$ ;
5 while  $r_1 < r$  do
6    $\forall u \in V_i, visited[u] \leftarrow false$  in parallel;  $b\_r \leftarrow b\_w$ ;
7    $len \leftarrow (end > start) ? (end - start) : \ell - (start - end)$ ;
8   if  $flag[r_2 + \text{block.id} \cdot L] == false$  then // clean
9     Obtain  $k_\Delta$  and  $k_\nabla$  w.r.t. Lemma 4.1;
10    parallel for  $j = \text{thread.id}; j < len; j += \text{blockDim.x}$  do
11       $u \leftarrow mrr[(start + j) \% \ell + \ell_b]$ ;  $visited[u] \leftarrow true$ ;
12       $mrr[(b\_w + j) \% \ell + \ell_b] \leftarrow u$ ;
13    parallel for  $j = \text{thread.id}; j < k_\Delta; j += \text{blockDim.x}$  do
14      Atomically sample  $u \in V_i$  with  $visited[u] == false$  and set
15       $visited[u] \leftarrow true$ ;
16       $mrr[(b\_w + len + j) \% \ell + \ell_b] \leftarrow u$ ;  $atomicAdd(cnt[u], 1)$ ;
17       $\_\_syncthreads()$ ;
18      Probe( $G_i, k_\Delta, b\_r + len, b\_w, mrr, visited, cnt$ );
19      parallel swap  $k_\Delta$  and  $k_\nabla$  roots to after previous  $k_{i-1}$  roots;
20  else // polluted
21    parallel for  $j = \text{thread.id}; j < k_i; j += \text{blockDim.x}$  do
22      if  $j < k_{i-1}$  then
23         $u \leftarrow mrr[(start + j) \% \ell + \ell_b]$ ;  $visited[u] \leftarrow true$ ;
24         $\_\_syncthreads()$ ;
25      if  $u \notin V_i$  or  $j \geq k_{i-1}$  then
26        Atomically sample  $u \in V_i$  with  $visited[u] == false$  and
27        set  $visited[u] \leftarrow true$ ;
28         $mrr[(b\_w + j) \% \ell + \ell_b] \leftarrow u$ ;  $atomicAdd(cnt[u], 1)$ ;
29         $\_\_syncthreads()$ ;
30      Probe( $G_i, k_i, b\_r, b\_w, mrr, visited, cnt$ );
31       $r_2 \leftarrow (r_2 + 1) \% r$ ;  $start \leftarrow offset[r_2 + \text{block.id} \cdot L]$ ;
32       $end \leftarrow offset[r_2 + 1 + \text{block.id} \cdot L]$ ;
33       $offset[r_1 + \text{block.id} \cdot L] \leftarrow b\_r$ ;  $r_1 \leftarrow r_1 + 1$ ;
34       $offset[r_1 + \text{block.id} \cdot L] \leftarrow b\_w$ ;

```

Algorithm 5: The Select Kernel

Input: Batch size b , cnt , mrr , $offset$, $flag$, θ_i .

```

1  $S_i \leftarrow \emptyset$ ;  $\Lambda(S_i) \leftarrow 0$ ;  $flag[\cdot] \leftarrow false$ ;  $cnt'[\cdot] \leftarrow cnt[\cdot]$ ;
2 while  $|S_i| < b$  do
3    $u \leftarrow \arg\max_{v \in V_i} \frac{cnt'[v]}{c(v)}$  by parallel reduction on GPU;
4    $S_i \leftarrow S_i \cup \{u\}$ ;  $\Lambda(S_i) \leftarrow \Lambda(S_i) + cnt'[u]$ ;
5   Kernel Discover( $cnt', mrr, offset, flag, \theta_i, \{u\}$ );
6   Derive  $\Lambda^l(S_i)$  and  $\Lambda^u(S_i^o)$ ;
7   if  $\Lambda^l(S_i) > \rho_b(1 - \hat{\epsilon})\Lambda^u(S_i^o)$  then  $\{cnt[\cdot] \leftarrow cnt'[\cdot]; \text{return } S_i\}$ 
8   else return  $\emptyset$ ;

```

cnt' (since this attempt may be rejected), and set all entries of $flag$ to false, as no mRR-set is covered before the first selection. In each iteration (Lines 2–5), we perform a GPU parallel reduction to select the node $u \in G_i$ with the maximum coverage-to-cost ratio, add u to S_i , and update $\Lambda(S_i)$. We then invoke the Discover kernel (Algorithm 6) to mark mRR-sets containing u as covered and decrement the corresponding coverage counts cnt' . After b seeds are selected, we test if S_i is qualified by computing the lower bound $\Lambda^l(S_i)$ of its coverage and the upper bound $\Lambda^u(S_i^o)$ for the optimal seed batch. If $\Lambda^l(S_i) > \rho_b(1 - \hat{\epsilon})\Lambda^u(S_i^o)$ holds, S_i is accepted, we overwrite cnt with cnt' , and return S_i (Line 7); otherwise, we return empty (Line 8), indicating that additional mRR-sets are needed.

In Algorithm 6, given a node set A (equal to $\{u\}$ in the Select kernel), Discover identifies all mRR-sets covered by A and updates the coverage counts of the nodes in those mRR-sets. In Section 4.4,

Algorithm 6: The Discover Kernel

Input: cnt , mrr , $offset$, $flag$, θ_i , node set A .

```

1  $r \leftarrow \theta_i / \text{gridDim.x} + \mathbb{I}_{\{\theta_i \% \text{gridDim.x} > \text{block.id}\}}$ ;  $r_1 \leftarrow 0$ ;
2 while  $r_1 < r$  do
3    $start \leftarrow offset[r_1 + \text{block.id} \cdot L]$ ;
4    $end \leftarrow offset[r_1 + 1 + \text{block.id} \cdot L]$ ;
5    $len \leftarrow (end > start) ? (end - start) : \ell - (start - end)$ ;
6   if  $flag[r_1 + \text{block.id} \cdot L] == false$  then
7     parallel for  $j = \text{thread.id}; j < len; j += \text{blockDim.x}$  do
8       if  $mrr[(start + j) \% \ell + \text{block.id} \cdot \ell] \in A$  then
9          $flag[r_1 + \text{block.id} \cdot L] \leftarrow true$ ;
10        if  $flag[r_1 + \text{block.id} \cdot L] == true$  then break;
11       $\_\_syncthreads()$ ;
12    parallel for  $j = \text{thread.id}; j < len; j += \text{blockDim.x}$  do
13       $u \leftarrow mrr[(start + j) \% \ell + \text{block.id} \cdot \ell]$ ;
14       $atomicSub(cnt[u], 1)$ ;
15       $\_\_syncthreads()$ ;
16       $r_1 \leftarrow r_1 + 1$ ;

```

Algorithm 7: GAAS

Input: Graph G , target η , estimation error ϵ , batch size b , cost $c(\cdot)$.
Output: Seed set S .

```

1  $i \leftarrow 1$ ,  $G_i \leftarrow G$ ,  $\eta_i \leftarrow \eta$ ,  $\rho_b \leftarrow 1 - (1 - 1/b)^b$ ,  $\theta_0 \leftarrow 0$ ,  $S \leftarrow \emptyset$ ;
2 Allocate and initialize arrays  $mrr$ ,  $offset$ ,  $flag$ , and  $cnt$  on GPU;
3 while  $\eta_i > 0$  do
4   Get  $\theta_i$  and  $\theta_{\max}$ ;
5   if  $\theta_{i-1} > 0$  then
6     ParallelUpdate( $G_i, cnt, mrr, offset, \theta_{i-1}, flag$ );
7      $S_i \leftarrow \text{Kernel Select}(b, cnt, mrr, offset, \theta_{i-1}, flag)$ ;
8     if  $S_i \neq \emptyset$  then  $\{\theta_i \leftarrow \theta_{i-1}; \text{goto Line 15}\}$ 
9     else  $\theta_i \leftarrow 2\theta_{i-1}$ ;
10    while  $\theta_i < \theta_{\max}$  do
11      ParallelGen( $G_i, cnt, mrr, offset, \theta_i, \theta_i$ );
12       $S_i \leftarrow \text{Kernel Select}(b, cnt, mrr, offset, \theta_i, flag)$ ;
13      if  $S_i \neq \emptyset$  then break;
14       $\theta_{i-1} \leftarrow \theta_i$ ;  $\theta_i \leftarrow 2\theta_i$ ;
15    Diffuse from  $S_i$ , and obtain the activated nodes  $A_i$ ;
16    Kernel Discover( $cnt, mrr, offset, flag, \theta_i, A_i$ );
17     $S \leftarrow S \cup S_i$ ;  $G_{i+1} \leftarrow G_i \setminus A_i$ ;  $\eta_{i+1} \leftarrow \eta_i - |A_i|$ ;  $i \leftarrow i + 1$ ;

```

we have also used Discover to identify the mRR-sets covered, i.e., polluted by the activated nodes in A_{i-1} . In each thread block, we first obtain the number of mRR-sets r in its segment (Line 1), then iterate over them with r_1 from 0 to $r - 1$ (Lines 2–16). For each mRR-set indexed by r_1 , we first obtain its start and end positions from the $offset$ array and its length using the GmRR layout in Section 4.2 (Lines 3–5). If its flag is false (Line 6), meaning it is not yet covered, all threads in the block (Line 7) scan this mRR-set in parallel to check whether any node is in A (Line 8); once such a node is found, they set the flag to true and terminate the scan (Lines 9–10). All threads in the block then collaboratively scan the nodes in this mRR-set and decrement coverage counts in cnt (Lines 12–14). The loop repeats until all mRR-sets in the segment have been processed.

4.6 The Complete GAAS Algorithm and Analysis

Algorithm 7 describes GAAS. Given G , node costs $c(\cdot)$, target η , estimation error ϵ , and batch size b (seeds per round), we initialize $i = 1$, G_i , η_i , S , and GmRR with arrays mrr , $offset$, $flag$, and cnt .

While $\eta_i > 0$, we first compute the required number of mRR-sets θ_i for round i and its upper bound $\theta_{\max}$. If mRR-sets from the previous round exist (i.e., $\theta_{i-1} > 0$), we update and reuse them for

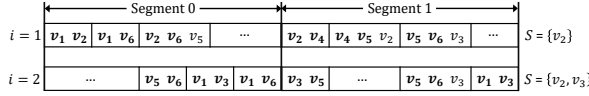


Figure 7: A running example of GAAS from round $i = 1$ to $i = 2$.

seed selection: we call `ParallelUpdate` to update all existing mRR-sets in parallel (Line 6), and then invoke `Select` to obtain b seeds S_i (Line 7). If S_i is non-empty (Line 8), the current mRR-sets suffice and we proceed directly to Line 15 without further generation; otherwise, we set $\theta_i \leftarrow 2\theta_{i-1}$ (Line 9) and continue. In this loop, we progressively enlarge the mRR-set collection until either $S_i \neq \emptyset$ or θ_i reaches $\theta_{\max}$. In each iteration, we invoke `ParallelGen` to get additional mRR-sets (Line 11), then run `Select` on the enlarged collection (Line 12); if S_i remains empty, we double θ_i and repeat.

Once a non-empty batch S_i is obtained, we simulate diffusion from S_i to obtain the activated nodes A_i (Line 15). We call `Discover` (Line 16) to mark polluted mRR-sets in *flag* for the next round and to decrement *cnt* so their contributions are removed. Finally, we update $S \leftarrow S \cup S_i$, construct G_{i+1} , derive η_{i+1} (Line 17), and proceed to the next round until the overall influence target is met.

In practice, the required number of mRR-sets usually decreases over rounds, so we periodically clear all segments and regenerate mRR-sets for a round when the remaining influence threshold is halved compared with previous regeneration round. This occasional regeneration discards excess mRR-sets, and its cost is amortized by the computational savings from updates in most rounds.

Example 4.5. Figure 7 shows, from input graph G in Figure 2(a), how mRR-sets are stored, updated, and used in GmRR across rounds. GmRR has two segments with size $\ell = 10$. In round $i = 1$, `ParallelGen` generates six mRR-sets using two thread blocks on $G_1 = G$, with segment 0 and 1 each storing three mRR-sets. The roots of the mRR-sets are in bold. In round 1, `Select` chooses $S = \{v_2\}$ as seed, which activates v_2 and v_4 under the realization in Figure 2(b). In round $i = 2$, `ParallelUpdate` updates the mRR-sets from round $i = 1$ on G_2 in Figure 5(a), using the update techniques in Section 4.4. For example, the mRR-set containing $\{v_5, v_6, v_3\}$ in round $i = 2$ is inherited from round $i = 1$. These updated mRR-sets are then used by `Select` to choose v_3 as seed, resulting in seed set $S = \{v_2, v_3\}$. Thus, we update and reuse mRR-sets across rounds to reduce overhead.

The theoretical guarantee of GAAS is given in Theorem 4.6, where π^o is the optimal policy that incurs the same per-round seed cost as GAAS. The approximation factor scales linearly with $\ln \eta$, improving the guarantees of [14, 39] by a factor of $\ln \eta$.

THEOREM 4.6. *The cost of GAAS reaching η approximates that of π^o in the form: $\mathbb{E}[c(\text{GAAS})] \leq \frac{2\ln \eta}{(1-\epsilon)(1-1/e)\rho_b} \mathbb{E}[c(\pi^o)] + b + 1$.*

PROOF SKETCH. Following the idea of [11], we consider a dual problem of AMCSS and analyze the approximation ratio of GAAS for the truncated influence achieved under an arbitrary budget B . By setting B in terms of $c(\pi^o)$, we show that the truncated influence is at least η with probability $1 - 1/\eta$. In the remaining case, which occurs with probability $1/\eta$, the cost to reach η is at most η . This yields the stated bound on the expected cost. $\square$

We show that the time complexity of update and reuse θ_{i-1} mRR-sets in round i is asymptotically smaller than that of generating them from scratch, and derive the space complexity of GAAS.

Table 1: Datasets information. ($K = 10^3, M = 10^6$)

Name	n	m	Avg. deg	Type
DBLP	317K	1.05M	6.62	undirected
YouTube	1.13M	2.99M	5.27	undirected
Flickr	2.30M	33.1M	14.4	directed
LiveJournal	4.85M	68.5M	14.1	directed

THEOREM 4.7. *Let α be the ratio of the estimated influence of A_{i-1} to the optimal influence under the same cost $c(S_{i-1})$. Updating θ_{i-1} mRR-sets in round i takes $O\left(\frac{ab(m_i+n_i)\ln n_{i-1}}{\epsilon^2} \cdot \frac{\mathbb{E}[\hat{f}(A_{i-1}|S_{i-1})]}{\eta_i}\right)$ time, where $\mathbb{E}[\hat{f}(A_{i-1}|S_{i-1})]/\eta_i < 1$, lower than generating the same number of mRR-sets from scratch that takes $O\left(\frac{ab(m_i+n_i)\ln n_i}{\epsilon^2}\right)$ time.*

PROOF SKETCH. Updating clean mRR-sets only requires new reverse BFS from new roots, which is equivalent to generating partial mRR-sets, while updating polluted mRR-sets is nearly as costly as generating new ones. Thus, we compute the expected numbers of clean and polluted mRR-sets, multiply by their respective update costs, and sum to obtain the total time complexity in round i . $\square$

Note that GPU parallelism is a practical factor and is thus omitted from the time complexity analysis, but evaluated in experiments.

THEOREM 4.8. *GAAS has expected space $O(m + \max_i \frac{abn_i \ln n_i}{\epsilon^2})$.*

PROOF SKETCH. In round i , the dominant cost is storing θ_i mRR-sets. Hence, the expected space is $O(\theta_i \cdot \mathbb{E}[|R|])$, where $|R|$ denotes the size of a random mRR-set in round i . Taking the maximum over rounds and adding graph storage gives the space complexity. $\square$

Discussion. When GPU memory becomes a bottleneck, GAAS is possible to be extended to multi-GPUs. The memory overhead mainly comes from storing mRR-sets, besides the graph G . The GmRR structure uses a segment-based layout, where each GPU thread block operates on its own dedicated segment. This design naturally extends to multi-GPUs by maintaining one local GmRR instance per GPU to generate and update mRR-sets locally. For seed selection, only the statistics derived from the GmRR instances need to be synchronized across GPUs, rather than the mRR-sets. If G is small enough, it can be replicated across GPUs to reduce cross-GPU communication. If G is large, it can be partitioned and distributed across GPUs, and reverse BFS for mRR-sets is performed in a distributed manner. We leave detailed investigation to future work.

5 Experiments

We evaluate GAAS and baselines on a server with an Intel Xeon Gold 6226R 2.90GHz CPU with 16 physical cores, 384 GB RAM, and an NVIDIA RTX 3090 GPU with 24 GB of memory.

5.1 Experimental Setup

Datasets. We adopt four datasets used in previous work [12, 19, 22, 39, 46]. The statistics are in Table 1. These datasets represent diverse social networks from real world and are available at [27]. The large dataset, LiveJournal, contains over 4.85 million nodes and 68.5 million edges, which is the largest network evaluated in [39]. **Baselines.** ASTI originally handles uniform seed cost, and is single-threaded on CPU [39]. We extend it to handle non-uniform costs by selecting seeds with the largest influence-to-cost ratios in each round. We also implement a CPU-parallel version, PASTI-CPU, parallelizing all applicable computations in ASTI using OpenMP [8] utilizing all CPU cores. Moreover, we include a GPU-parallel ASTI version, PASTI-GPU, which adapts the GPU-based techniques from

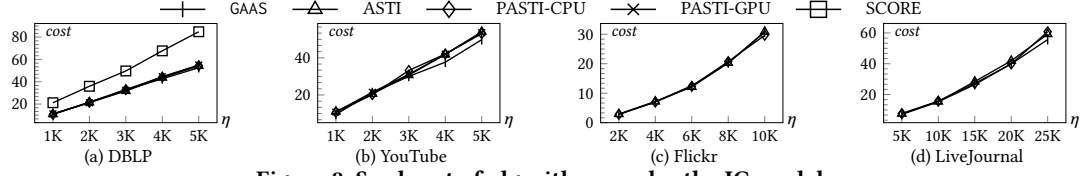


Figure 8: Seed cost of algorithms under the IC model.

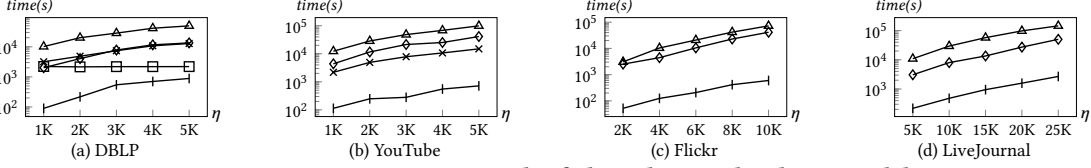


Figure 9: Running time in seconds of algorithms under the IC model.

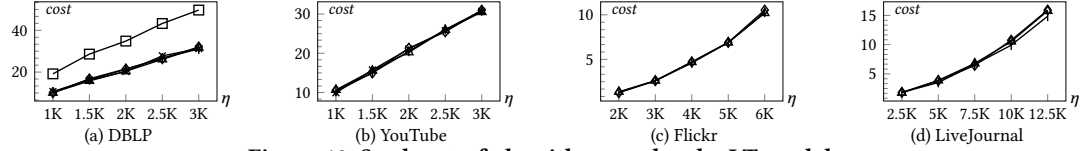


Figure 10: Seed cost of algorithms under the LT model.

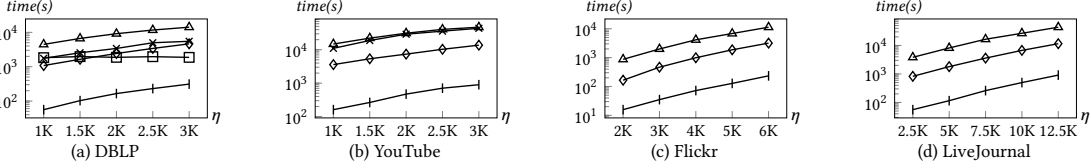


Figure 11: Running time in seconds of algorithms under the LT model.

[37] (originally designed for single-root RR-set generation in Influence Maximization). PASTI-GPU maintains mRR-sets in a contiguous way in the global GPU memory. Finally, we compare with SCORE, a non-adaptive MCSS method that provides a probabilistic guarantee on η in any realization. We configure SCORE with high confidence ($p = 0.99$) to align with the stopping criterion of adaptive methods reaching η with probability 1.0.

Parameters. We use consistent parameter settings across all tested baselines and GAAS. Following [12, 19, 22, 39], we set the mRR-set estimation error $\epsilon = 0.5$ and failure probability $\delta = 1/n_i$ per round. The diffusion probability of any edge (u, v) is $p(u, v) = 1/|N_{in}(v)|$, and the cost of a node u is $c(u) = c_0 + 0.01 \cdot |N_{out}(u)|$ with $c_0 = 0.01$ to avoid zero-cost nodes. We evaluate all methods under the IC and LT models, averaging results over 10 realizations. Under the IC model, we select 4 nodes per batch on DBLP and 16 nodes per batch on larger datasets to ensure the single-threaded baseline ASTI completes within two days. η ranges from 1K to 5K on DBLP and YouTube, from 2K to 10K on Flickr, and from 5K to 25K on LiveJournal, reflecting their different scales. Under the LT model, we fix the batch size $b = 4$ for all datasets, and vary η ranges according to dataset size. The ranges of η vary from 1K to 3K on DBLP and YouTube, from 2K to 6K on Flickr, and from 2.5K to 12.5K on LiveJournal. In GAAS, when updating mRR-sets, we set the number of groups to $\beta = 8$ for load balancing, as determined by ablation study in Section 5.4. We configure GAAS and all parallel baselines to utilize all available CPU cores and GPU resources.

5.2 Seed Cost and Efficiency under the IC model

AMCSS aims to minimize seed cost while reaching the target influence η . Thus, efficiency is comparable only when methods reliably

attain η with low seed cost. By the theoretical guarantees of GAAS and the adaptive baselines, all adaptive methods always attain η (100% attainment). We therefore omit attainment plots (all equal to 1.0) and focus on comparing the *seed cost* and *running time*.

Seed Cost. Figure 8 reports the seed cost to reach various η under the IC model. All adaptive methods yield similar low costs and require substantially less seed cost than the non-adaptive SCORE on DBLP. Because GAAS and the adaptive ASTI baselines all leverage the mRR-set estimation framework, they have similar estimation accuracy and thus comparable seed cost. Nonetheless, we observe that GAAS achieves slightly lower cost, especially on YouTube and LiveJournal. This is because, when updating mRR-sets in adaptive rounds, GAAS deliberately retains all previous mRR-sets, even though the number of required mRR-sets typically decreases over rounds, allowing it to exploit more mRR-sets than regenerating-based methods and make higher-quality seeding decisions. The higher cost of SCORE arises from its non-adaptive nature: it cannot leverage intermediate results and therefore selects more nodes to ensure η with the required probability. SCORE runs out of memory (OOM) on the other three larger datasets, since it depends on graph sketches, and the sketch size grows rapidly as the graph scales.

Running Time. Given the minimal and similar seed costs, we compare the efficiency. Figure 9 reports the running time (y-axis in log scale). GAAS consistently runs faster than all baselines, often by more than an order of magnitude. For all methods, running time increases with larger η since higher thresholds require more rounds. Compared with the strongest GPU-based baseline PASTI-GPU, GAAS achieves speedups of up to 33.9 $\times$ and 24.2 $\times$ on DBLP and YouTube, respectively. These gains stem from our GmRR structure,

which enables in-segment mRR-set generation and update directly on GPU. In contrast, PASTI-GPU runs OOM on the larger Flickr and LiveJournal for two main reasons. First, it relies heavily on dynamic memory allocation, which leads to severe memory fragmentation. Second, it temporarily buffers mRR-sets being generated in global memory, effectively doubling peak memory usage in the worst case. GAAS avoids these issues by combining GmRR with carefully designed parallel mRR-set generation and update kernels, which use global memory and thread blocks more efficiently. On the two largest datasets, GAAS achieves substantial speedups over the strongest CPU baseline PASTI-CPU. On Flickr, GAAS runs up to $68.9\times$ faster than PASTI-CPU when $\eta = 10K$, and on LiveJournal it attains more than $10\times$ speedup. For SCORE on DBLP, its running time in Figure 9(a) is less informative because it incurs significantly higher seed cost in Figure 8(a). Even so, GAAS remains much faster than SCORE with lower seed cost. SCORE runs OOM on other datasets.

5.3 Seed Cost and Efficiency under the LT model

Under the LT model, GAAS is also faster than existing methods, often by an order of magnitude, while maintaining low seed cost.

Seed Cost. Figure 10 reports the seed cost under the LT model. The trends are consistent with those in Figure 8. For each dataset and influence threshold η , all adaptive methods incur similar costs, whereas the non-adaptive SCORE requires substantially higher cost on DBLP and runs out of memory on larger datasets. Overall, GAAS achieves comparable or lower cost than all baselines, with its advantage increasing for larger η , where additional rounds allow it to reuse and update existing mRR-sets instead of regenerating them, leading to more accurate truncated influence estimation.

Time Efficiency. Figure 11 reports the running time under the LT model. The results show that GAAS consistently outperforms all baseline methods across all evaluated datasets. Note that SCORE and PASTI-GPU run OOM on the two largest datasets and thus do not produce results there. Across all influence targets η where the baselines succeed, GAAS achieves more than $10\times$ speedup over the strongest competitor. In particular, GAAS attains speedups of up to $19.2\times$ on DBLP, $22.3\times$ on YouTube, $14.6\times$ on Flickr, and $15.7\times$ on LiveJournal compared to the fastest baseline. These substantial performance gains arise from our carefully designed GPU optimizations, including independent inter-block mRR-set generation and the efficient selective reuse of clean mRR-sets, which together minimize redundant computation and fully exploit GPU parallelism.

5.4 Experimental Analysis

Time Breakdown and the Effect of ParallelUpdate. A key design in GAAS is to update and reuse mRR-sets from previous rounds by the GPU kernel `ParallelUpdate` instead of regenerating. To quantify the efficiency benefit, we compare GAAS with a variant $GAAS_{gen}$ that regenerates mRR-sets from scratch in every round on GPU. For each method, we decompose the running time into three components: mRR-set generation, mRR-set update, and others (including seed selection). In Figure 12, we report the time breakdown of GAAS and $GAAS_{gen}$ under the IC model. The bar with $\star$ represents GAAS, while the bar without $\star$ represents $GAAS_{gen}$. It shows that GAAS with `ParallelUpdate` significantly reduces time spent on mRR-set generation: in $GAAS_{gen}$, mRR-set generation dominates

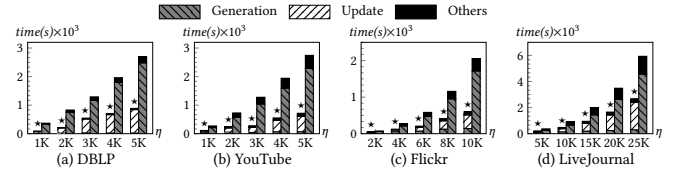


Figure 12: Runtime breakdown under the IC model; for each η , the left bar with $\star$ is GAAS and the right bar is $GAAS_{gen}$.

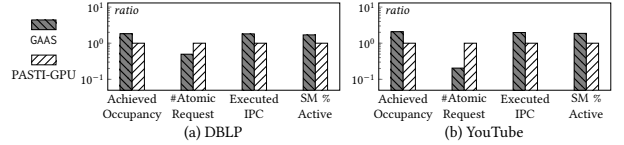


Figure 13: GPU profiling analysis relative to PASTI-GPU.

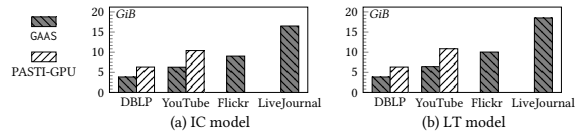


Figure 14: GPU memory comparison with PASTI-GPU.

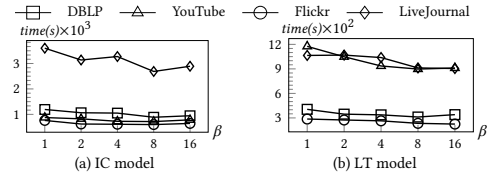


Figure 15: Vary β under largest tested η across datasets.

the runtime, whereas in GAAS it becomes negligible and the total running time is much lower. This confirms that GAAS effectively reuses existing mRR-sets by updating mRR-sets in parallel on the GPU. The results under the LT model are similar and deferred to the technical report [2]. GAAS outperforms $GAAS_{gen}$ in efficiency, confirming the power of our proposed mRR-set update kernel.

GPU Profiling. We use NVIDIA Nsight Compute [1] to quantify the efficiency of our GPU method via standard GPU performance metrics. Figure 13 reports the GPU occupancy level, instructions executed per cycle (IPC), and streaming multiprocessor (SM) active percentage—where higher values indicate better performance—as well as the number of atomic operations, where lower values are preferred for efficiency, comparing GAAS with PASTI-GPU on DBLP and YouTube under the IC model. Clearly, GAAS achieves better GPU occupancy, higher IPC, and higher SM throughput while requiring fewer atomic operations than PASTI-GPU. For example, GAAS attains $1.8\times$ and $2.1\times$ higher occupancy than PASTI-GPU. These demonstrate that GAAS utilizes GPU resources more efficiently, owing to our algorithmic designs and performance optimizations.

GPU Memory Usage. We compare GAAS with PASTI-GPU in Figure 14 under IC and LT models. It shows that GAAS uses less GPU memory than PASTI-GPU on DBLP and Youtube, while PASTI-GPU runs out of memory on the larger Flickr and LiveJournal, with reasons discussed in Section 5.2. GAAS uses the proposed GmRR structure with carefully designed parallel mRR-set generation and update kernels, resulting in efficient memory usage on GPU.

Vary β for load balancing. We vary $\beta \in \{1, 2, 4, 8, 16\}$ to study its impact under both the IC and LT models. When $\beta = 1$, no load

balancing is applied. In Figure 15, observe that the running time generally decreases as β increases from 1 to 8. However, when β increases from 8 to 16, the running time starts to increase, because the overhead of finer-grained grouping and block-level synchronization outweighs the benefits. Therefore, we set $\beta = 8$.

6 Conclusion

We present a GPU-accelerated AMCSS method GAAS by integrating novel algorithmic ideas with GPU-aware system design. We design a GPU-tailored mRR-set structure GmRR, and build a ParallelGen kernel for mRR-set generation, a ParallelUpdate kernel that reuses and updates previous mRR-sets via theoretically sound rules and circular in-segment updates with load balancing, and a Select kernel for parallel seed selection. Experiments on real-world graphs show that GAAS is over an order of magnitude faster than the base-lines, while achieving comparable or lower seed cost. [We plan to extend GAAS to distributed or multi-GPU settings as future work.](#)

7 Notations

Table 2: Frequently used notations.

$G = (V, E)$	social network with users V and edges E
$G_i = (V_i, E_i)$	residual graph in round i
n, m	number of users and edges
$N_{\text{in}}(u), N_{\text{out}}(u)$	in-/out-neighbors of u
$p(e), p(u, v)$	diffusion probability of edge e or (u, v)
θ_i	# of mRR-sets used in round i
k_i	# of roots in mRR-sets in round i
ϵ	estimation error of mRR-sets
η	target influence threshold
η_i	remaining target influence in round i
S_i	seed set selected in round i
A_i	activated nodes in round i
$c(v)$	cost of seeding node v
$I(S)$	random influence spread of S
ϕ, Φ	specific / random realization
$\Gamma_i(S)$	truncated influence of S in G_i
$\tilde{\Gamma}(S)$	estimated truncated influence
π	adaptive seed selection policy

8 Missing Proofs

8.1 Proof of Lemma 4.1

We first prove that the nodes in a clean mRR-set R can be preserved. Then, we show that our way of adding roots is correct.

(1) Note that the root distribution of a clean mRR-set R is the same as selecting the roots in the new residual graph. Let the original root set of R be K . The probing result that generates R can be represented as a tree structure T . The fact that R is not polluted indicates that the frontier of the reverse BFS from K (i.e., the leaves) did not probe activated nodes or the activation failed. For each probe, the probability of its success is not affected by whether it is probed in G_{i-1} or G_i . Thus, the probability of reaching the nodes in R is the same as reaching them in G_i . Similarly, for the nodes in V_i that were probed but failed, the probability is also the same. For the failed probes w.r.t. nodes in $V_{i-1} \setminus V_i$, since they are not in

V_i , they will not be in R as well. Thus, the composite of R will not change in a new generation in G_i .

Then, we show that when new roots are selected, the nodes in R can still be preserved. The selection of new seeds results in two cases. For the roots that were in R , we argue that the composite of R will not change. For each node $u \in R$, there must be a path from one of the root (e.g. r) to u . When some node v in the path is promoted to be a root. The path from r to v will not change, but the nodes after v (denoted as $O = \{o_1, o_2, \dots, o_l\}$, where l is the length) may be traversed by some other nodes in R . We can prove that the nodes in O will still be in R , by induction. For the first node o_1 , it may be probed by a new node other than v . If the probe is successful, o_1 will stay in R . Otherwise, at least o_1 can be traversed by v . Either way, o_1 will still be in R . Similarly, assuming o_i is in R , we can prove that o_{i+1} is still in R . Thus, the nodes in R will not change.

Thus, the elements in R have the same distribution as an mRR-set generated from scratch in the new residual graph.

(2) To validate our way of adding roots, we first invoke Lemma 4.3, which shows that by replacing the activated roots with randomly selected ones, the root set is equal to a newly selected root set. Then, in this lemma, we do not consider the case where there are activated roots, since we know R is clean.

Assume a random mRR-set R with k_{i-1} roots K , which needs $\Delta = k_i - k_{i-1}$ new roots. When adding new roots into R , to mimic the generation of fresh mRR-sets, we only avoid selecting the roots already in R . Then, some non-root nodes in R may be selected as a root. To accommodate this case, we select some new roots from $R \setminus K$ and some from $V_i \setminus R$. According to Alg. 4, to ensure a uniform sampling, the numbers of roots we select from $R \setminus K$ and $V_i \setminus R$ are proportional to their cardinalities. That is, $\Delta \cdot \frac{|R| - k_{i-1}}{n_i - k_{i-1}}$ roots are selected from $R \setminus K$, and $\Delta \cdot \frac{n_i - |R|}{n_i - k_{i-1}}$ from $V_i \setminus R$. Let the roots selected from $R \setminus K$ (resp. $V_i \setminus R$) be D_1 (resp. D_2), and $D = D_1 \cup D_2$ ($D \cap K = \emptyset$, $D \subseteq V_i$, and $|D| = k_i - k_{i-1} = \Delta$).

We first prove that the k_{i-1} roots from the previous round can be reused. To see it, we consider an intermediate way of root supplementation. That is, we select Δ roots from $V_i \setminus K$. For any set of k_i nodes $T \subseteq V_i$, the probability that the new root set equals to T is $\frac{C_{k_i}^{k_{i-1}}}{C_{n_i}^{k_{i-1}}} \frac{C_{n_i - k_{i-1}}^{k_i - k_{i-1}}}{C_{n_i - k_{i-1}}^{k_i - k_{i-1}}}$. By similar derivation in Section 8.2, the probability is equal to selecting k_i fresh roots from V_i directly.

Next, we show that the probability of a node being selected as a root by Lemma 4.1 is the same as the way in Lemma 4.3, where the probability is $\Delta / (n_i - k_{i-1})$. As for our way of selecting roots from $R \setminus K$ and $V_i \setminus R$ separately, the probability of a node $u \in R \setminus K$ being selected as a root is $\frac{1}{|R| - k_{i-1}} \cdot \Delta \cdot \frac{|R| - k_{i-1}}{n_i - k_{i-1}} = \frac{\Delta}{n_i - k_{i-1}}$. For a node $u \in V_i \setminus R$, the probability is $\frac{1}{n_i - |R|} \cdot \Delta \cdot \frac{n_i - |R|}{n_i - k_{i-1}} = \frac{\Delta}{n_i - k_{i-1}}$. Thus, the probability of any node in both $R \setminus K$ and $V_i \setminus R$ being selected as a root is the same as selecting directly from $V_i \setminus K$.

8.2 Proof of Lemma 4.3

For ease of notation, we write $\Delta = k_i - k_{i-1}$. Consider an arbitrary set K of k_i nodes in V_i . When generating a fresh mRR-set, the probability of K being selected as the roots is $1/C_{n_i}^{k_i}$, which is uniformly distributed. To establish the lemma, we only need to prove that

the root set extended from the previous round is also uniformly distributed, corresponding with K with probability $1/C_{n_i}^{k_i}$.

Let $K_{i-1} = K_1 \cup K_2$ ($K_1 \cap K_2 = \emptyset$), where K_1 is the set of inactivated roots preserved from V_{i-1} , and K_2 is the set of roots activated and deleted in the last round. According to Alg. 4, to obtain k_i roots, we need to select $\Delta' = \Delta + |K_2|$ roots from V_i . Combining the Δ' roots and the preserved roots K_1 , we obtain the new root set K' . The probability that K' corresponds with K is

$$\begin{aligned} \Pr[K' = K] &= \frac{C_{k_i}^{|K_1|}}{C_{n_i}^{|K_1|} C_{n_i - |K_1|}^{\Delta'}} \\ &= \frac{k_i!}{|K_1|!(k_i - |K_1|)!} \cdot \frac{|K_1|!(n_i - |K_1|)!\Delta'!(n_i - |K_1| - \Delta')!}{n_i!(n_i - |K_1|)!} \\ &= \frac{k_i!(n_i - k_i)}{n_i!} = 1/C_{n_i}^{k_i}, \end{aligned}$$

where the second equality is due to $\Delta' + |K_1| = k_i$.

As can be seen, after applying Alg. 4 to update the roots of an mRR-set, the distribution of the roots is the same as directly selecting nodes from the new residual graph, both uniformly distributed.

8.3 Concentration Inequalities

Let Y_i be an indicator random variable, which takes value 1 if the i -th randomly generated mRR-set is covered by the seed set S , and takes value 0 otherwise. On this basis, we present Lemma 8.1, which is proven in [40].

LEMMA 8.1. *Let $\bar{Y} = \frac{1}{\theta} \sum_{i=1}^{\theta} Y_i$ denote the average of the indicator random variables. For any $\lambda \geq 0$, we have*

$$\begin{aligned} \Pr \left[\mathbb{E}[\bar{Y}] \cdot \theta < \left(\sqrt{\bar{Y} \cdot \theta + \frac{2\lambda}{9}} - \sqrt{\frac{\lambda}{2}} \right)^2 - \frac{\lambda}{18} \right] &\leq e^{-\lambda}, \\ \Pr \left[\mathbb{E}[\bar{Y}] \cdot \theta > \left(\sqrt{\bar{Y} \cdot \theta + \frac{\lambda}{2}} + \sqrt{\frac{\lambda}{2}} \right)^2 \right] &\leq e^{-\lambda}. \end{aligned}$$

The derivation of Theorem 4.6 relies on Lemma 8.2 and Lemma 8.3.

LEMMA 8.2. *Given a budget B and a seed set S selected by GAAS, if $c(S) = B$ (i.e., the budget is used up exactly), then the coverage of S approximates the seed set S^0 with the largest coverage by $\Lambda(S) \geq \rho_b \Lambda(S^0)$, which holds with probability at least $1 - 1/n_i$.*

LEMMA 8.3. *In each round, the seed set S returned by GAAS approximates the optimal seed set S^0 that takes the same cost as S in the following form*

$$\mathbb{E}[\Gamma(S|S_{i-1})] \geq \rho_b(1 - 1/e)(1 - \epsilon)\mathbb{E}[\Gamma(S^0|S_{i-1})].$$

To prove the approximation ratio of GAAS, we consider a dual problem of AMCSS, called truncated influence maximization (TIM). Specifically, we are given some budget B , and select b seeds in each round. The influence considered is truncated by η . In this way, we connect TIM and AMCSS. The aim is to maximize the truncated influence.

To solve TIM, we adopt the same seed selection process as GAAS in each round. Nevertheless, the stopping condition is not reaching η , but the budget is used up. For the last round of seed selection, the remaining budget B' may be not enough (i.e., $B' < c(S)$). In this

case, we select S with probability $B'/c(S)$, to respect the budget constraint in expectation.

Let the policy described above be π^d . To derive a general performance guarantee for π^d , we consider a general budget l , which can be larger than B . Given the approximation ratio of $\rho_b(1 - 1/e)(1 - \epsilon)$ in each round of seed selection by Lemma 8.3, we can derive the approximation ratio of π^d in Theorem 8.4 by applying Theorem 38 in [14], which only relies on the adaptive submodularity and adaptive monotonicity of $\Gamma(\cdot)$ which are proven in [39]. That is, we have $\mathbb{E}[\Gamma(\pi^d)] \geq (1 - e^{-\alpha l/B})\mathbb{E}[\Gamma(\pi)]$, where $\alpha = \rho_b(1 - 1/e)(1 - \epsilon)$ and π is an arbitrary policy for TIM.

THEOREM 8.4. *Consider any user cost $c : V \rightarrow \mathbb{N}$ and any seed selection accuracy $0 \leq \alpha \leq 1$ in each round. For any policy π under budget B , we have*

$$\mathbb{E}[\Gamma(\pi^d)] \geq (1 - e^{-\alpha l/B})\mathbb{E}[\Gamma(\pi)].$$

8.4 Proof of Lemma 8.2

By previous work [41], we know that $\Lambda(\cdot)$ is a submodular set function, since it is actually the objective function of the set cover problem. Let S^* be the node set with the largest coverage. Note that S^* is different from S^0 , since the definitions of their optimality are different. And we have $\Lambda(S^*) \geq \Lambda(S^0)$. By Lemma 3 of [26], we have

$$\Lambda(S) \geq \left[1 - \prod_{j=1}^b \left(1 - \frac{c(s_j)}{B} \right) \right] \Lambda(S^*), \quad (1)$$

where s_j is the j -th seed selected into S . Note that the estimation error in Lemma 3 of [26] is set to 0, since the coverage of any node set in a collection of mRR-sets can be accurately counted.

By the arguments in the proof of Theorem 1 of [26], the approximation ratio in Ineq. 1 takes the minimum value when $c(s_j) = B/b, \forall j$. Thus, we have

$$\Lambda(S) \geq \left[1 - \prod_{j=1}^b \left(1 - \frac{1}{b} \right) \right] \Lambda(S^*) = \rho_b \Lambda(S^*) \geq \rho_b \Lambda(S^0).$$

8.5 Proof of Lemma 8.3

The proof idea is similar to the proof in [39]. We first clarify two concepts during the seed selection in GAAS, which involves multiple trials to select a seed. In each trial, GAAS proposes a candidate seed set, based on the coverages of nodes. If the candidate seed set is verified to be unqualified, GAAS discards this candidate set, and doubles the mRR-sets to select a new candidate set. Otherwise, GAAS returns the selected seed set as the final seed set.

GAAS contains two stopping conditions. First, the number of mRR-sets reaches $\theta_{\max}$; second, the candidate seed set is verified to be legitimate. We next show that, in any case, the seed set returned by GAAS is approximates the optimal seed set by at least $\rho_b(1 - 1/e)(1 - \epsilon)$.

For the first case $\theta \geq \theta_{\max}$, following similar derivation from [39, 40], we can see that $\mathbb{E}[\hat{\Gamma}(S|S_{i-1})] \geq (1 - \epsilon)\rho_b\mathbb{E}[\hat{\Gamma}(S^0|S_{i-1})]$ holds with probability $1 - \delta/3$. Further, we have

$$\mathbb{E}[\Gamma(S|S_{i-1})] \geq \mathbb{E}[\hat{\Gamma}(S|S_{i-1})]$$

$$\geq (1 - \epsilon)\rho_b\mathbb{E}[\hat{\Gamma}(S^0|S_{i-1})] \geq (1 - \epsilon)\rho_b(1 - 1/e)\mathbb{E}[\Gamma(S^0|S_{i-1})],$$

where the first and last inequalities are due to Ineq. (??).

For the second case, we consider the following two events for any seed set $S \subseteq V$,

$$E_1(S) : \mathbb{E}[\Lambda(S)] \geq \left(\sqrt{\Lambda(S) + \frac{2a_1}{9}} - \sqrt{\frac{a_1}{2}} \right)^2 - \frac{a_1}{18},$$

$$E_2(S) : \mathbb{E}[\Lambda(S)] \leq \left(\sqrt{\Lambda(S) + \frac{a_2}{2}} + \sqrt{\frac{a_2}{2}} \right)^2.$$

Then, by Lemma 8.1 and the definitions of a_1 and a_2 , we have

$$\Pr[-E_1(S)] \leq \delta/3HC_{n_i}^b,$$

$$\Pr[-E_2(S)] \leq \delta/3H.$$

Note that the seed set S returned by GAAS is random, and can be any node set with size b from V . Thus, we apply the total probability and derive

$$\begin{aligned} \Pr[\mathbb{E}[\Lambda(S) < \Lambda^l(S)]] &= \Pr[-E_1(S)] \\ &= \sum_{S' \subseteq V_i} \Pr[-E_1(S')] \cdot \Pr[S' = S] \\ &\leq \frac{\delta}{3H} \sum_{S' \subseteq V_i} \Pr[S = S'] \\ &= \delta/3H. \end{aligned} \quad (2)$$

For the optimal seed set S^o , the event $E_2(S^o)$ directly implies

$$E_2'(S^o) : \mathbb{E}[\Lambda(S^o)] \leq \left(\sqrt{\Lambda(S)/\rho_b} + \frac{a_2}{2} + \sqrt{\frac{a_2}{2}} \right)^2 = \Lambda^u(S^o),$$

since $\Lambda(S) \geq \rho_b \Lambda(S^o)$ by Lemma 8.2 and the r.s.h. of E_2 is increasing with $\Lambda(S)$. Note that S^o is unknown but certain. Thus, we do not need to apply the total probability and can write

$$\Pr[\mathbb{E}[\Lambda(S^o)] > \Lambda^u(S^o)] = \Pr[-E_2'(S^o)] \leq \Pr[-E_2(S^o)] \leq \delta/3H. \quad (3)$$

Combining the above analysis for S and S^o , we can see that, with probability $1 - 2\delta/3H$, the events in Ineqs. (2) and (3) do not hold, which lead to $\mathbb{E}[\Lambda(S)] \geq \Lambda^l(S)$ and $\mathbb{E}[\Lambda(S^o)] \leq \Lambda^u(S^o)$. Further, when GAAS returns S in the second case, we can derive

$$\begin{aligned} \mathbb{E}[\Gamma(S|S_{i-1})] &\geq \mathbb{E}[\hat{\Gamma}(S|S_{i-1})] = \mathbb{E}[\Lambda(S)]n_i/\theta \geq \Lambda^l(S)n_i/\theta \\ &\geq (1 - \hat{\epsilon})\rho_b \Lambda^u(S^o)n_i/\theta \geq (1 - \hat{\epsilon})\rho_b \mathbb{E}[\Lambda(S^o)]n_i/\theta \\ &= (1 - \hat{\epsilon})\rho_b \mathbb{E}[\hat{\Gamma}(S^o|S_{i-1})] \geq (1 - \hat{\epsilon})\rho_b (1 - 1/e) \mathbb{E}[\Gamma(S^o|S_{i-1})], \end{aligned} \quad (4)$$

where the first and last inequalities are due to Ineq. (??). The second and penultimate equalities are due to the definition of $\hat{\Gamma}(\cdot)$. The third and antepenult inequalities are due to Ineqs. (2) and (3). The middle fourth inequality is due to the stopping condition of GAAS. When η is reached, at most H rounds of seed selection are carried out. The probability that Ineq. (4) holds in any round is at least $1 - (2\delta/3H) \times H = 1 - 2\delta/3$.

Although Ineq. (4) fails with very small probability, we still take it into account to derive the performance guarantee of GAAS in expectation, which is taken w.r.t. the randomness ω in seed selection. That is

$$\begin{aligned} \mathbb{E}_\omega[\mathbb{E}[\Gamma(S|S_{i-1})]] &\geq (1 - \hat{\epsilon})(1 - \delta)\rho_b (1 - 1/e) \mathbb{E}[\Gamma(S^o|S_{i-1})] \\ &\geq (1 - \epsilon)\rho_b (1 - 1/e) \mathbb{E}[\Gamma(S^o|S_{i-1})]. \end{aligned}$$

This completes the proof.

8.6 Proof of Theorem 4.6

The proof idea is similar to [11]. We prove this theorem by showing that when we spend $(\ln \eta + 1)$ times more cost than the optimal policy, the influence threshold η can be reached with high probability.

Let π^o be the optimal policy. The expected cost of π^o is $\mathbb{E}[c(\pi^o)]$. Next, we apply Theorem 8.4 by letting $B = \mathbb{E}[c(\pi^o)]$ and $l = \frac{2 \ln \eta}{(1-\epsilon)(1-1/e)\rho_b} \mathbb{E}[c(\pi^o)]$, i.e., the ratio times optimal cost. Recall the influence obtained by π^d in Theorem 8.4 and let the arbitrary policy be π^o , we see that

$$\mathbb{E}[\Gamma(\pi^d)] \geq \left(1 - e^{-\frac{\rho_b(1-1/e)(1-\epsilon)l}{\mathbb{E}[c(\pi^o)]}} \right) \mathbb{E}[\Gamma(\pi^o)].$$

Plugging the value of l , we derive that

$$\mathbb{E}[\Gamma(\pi^d)] \geq \left(1 - e^{-2 \ln \eta} \right) \mathbb{E}[\Gamma(\pi^o)] = \eta - 1/\eta.$$

Note that $\mathbb{E}[\Gamma(\pi^d)]$ is an expected influence, which can be expressed as

$$\begin{aligned} \eta - 1/\eta &\leq \mathbb{E}[\Gamma(\pi^d)] = \sum_{x=1} \eta \Pr[\Gamma(\pi^d) = x] \\ &\leq \eta \Pr[\Gamma(\pi^d) = \eta] + (\eta - 1) \Pr[\Gamma(\pi^d) < \eta], \end{aligned}$$

which leads to $\Pr[\Gamma(\pi^d) = \eta] \geq 1 - 1/\eta$.

Note that the policy π^d designed for TIM selects seed sets in the same as GAAS in AMCSS, and the influence function is also truncated. Thus, the influence reached by π^d is the same as GAAS, before the last round of seed selection, where the seed set in π^d is selected probabilistically. To achieve at least the same influence as π^d , we only need to ask GAAS to select the seed set in the last round of π^d with certainty. Then, the cost spent by GAAS in the last round is also certain, resulting at most b more cost than π^d . That is, when the cost used by GAAS is $l + b$, we have $\Pr[\Gamma(\text{GAAS}) = \eta] \geq 1 - 1/\eta$.

On the other hand, with probability at most $1/\eta$, the influence of GAAS is smaller than η . In this case, at most η users are selected, resulting in a cost of η .

Summarizing the above analysis, we can derive the expected cost GAAS needed to reach η

$$\begin{aligned} \mathbb{E}[c(\text{GAAS})] &\leq (l + b)(1 - 1/\eta) + \eta \cdot 1/\eta \leq l + b + 1 \\ &= \frac{2 \ln \eta}{(1 - \epsilon)(1 - 1/e)\rho_b} \mathbb{E}[c(\pi^o)] + b + 1. \end{aligned}$$

This completes the proof.

8.7 Proof of Theorem 4.7

By Lemma 3.8 of [39], the time to generate an mRR-set in round i is $T_{\text{mRR}} = O\left(\frac{\max_{v \in V_i} \mathbb{E}[\hat{\Gamma}(v|S_{i-1})]}{\eta_i} m_i\right)$. Slightly extending Lemma 3.9 of [39], we can see that the number of mRR-sets needed in round i is $\theta_i = O\left(\frac{b \eta_i \ln n_i}{(\epsilon^2 \max_{S \subseteq V_i, |S|=b} \mathbb{E}[\hat{\Gamma}(S|S_{i-1})])}\right)$, where B_i is the cost spent by GAAS. Then, when ASTI is extended to the cost-aware case, its time complexity is $O\left(\frac{b(m_i + n_i) \ln n_i}{\epsilon^2} \frac{\max_{v \in V_i} \mathbb{E}[\hat{\Gamma}(v|S_{i-1})]}{\max_{S \subseteq V_i, |S|=b} \mathbb{E}[\hat{\Gamma}(S|S_{i-1})]}\right)$.

On the other hand, for GAAS framework, the time complexity to generate an mRR-set is still T_{mRR} . The revision in GAAS includes two

cases: generating new mRR-sets for polluted ones, and add roots for unpolluted ones.

For the first case, we need to know how many mRR-sets are expected to be polluted, which is the number of mRR-sets covered by A_{i-1} . Since the coverage is an unbiased estimation of truncated influence, in turn we can express the coverage by the truncated influence of A_{i-1} , i.e., $\theta_{i-1} \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})]/\eta_{i-1}$.

In the second case, for each unpolluted mRR-sets, we only need to add $k_i - k_{i-1}$ roots. By Alg. 4, we simulate at most $\Delta = k_i - k_{i-1}$ times reverse sampling from the added roots. The expected time of simulating the sampling is $\frac{k_i - k_{i-1}}{k_i} T_{\text{mRR}}$.

Then, the expected time to obtain θ_i mRR-sets is

$$\begin{aligned}
& \left[\frac{\theta_{i-1} \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})]}{\eta_{i-1}} + \left(\theta_{i-1} - \frac{\theta_{i-1} \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})]}{\eta_{i-1}} \right) \frac{\Delta}{k_i} \right] T_{\text{mRR}} \\
&= \frac{\theta_{i-1}}{\eta_{i-1}} \left[\left(\eta_i - \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})] \right) \left(1 - \frac{n_{i-1}}{\eta_{i-1}} \frac{\eta_{i-1} - |A_{i-1}|}{n_{i-1} - |A_{i-1}|} \right) \right. \\
&\quad \left. + \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})] \right] T_{\text{mRR}} \\
&\leq \frac{\theta_{i-1}}{\eta_{i-1}} \left[|A_{i-1}| \left(1 - \frac{\mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})]}{\eta_{i-1}} \right) + \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})] \right] T_{\text{mRR}} \\
&= O \left(\left(|A_{i-1}| + \mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})] \right) \frac{\theta_{i-1}}{\eta_{i-1}} T_{\text{mRR}} \right) \\
&= O \left(\left(\mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})] \right) \frac{\theta_{i-1}}{\eta_{i-1}} T_{\text{mRR}} \right) \\
&= O \left(\frac{b \ln n_{i-1} (\mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})])}{\epsilon^2 \max_{S \subseteq V_i, |S|=b} \mathbb{E}[\hat{\Gamma}(S|S_{i-1})]} T_{\text{mRR}} \right) \\
&= O \left(\frac{b(m_i + n_i) (\mathbb{E}[\hat{\Gamma}(A_{i-1}|S_{i-1})]) \ln n_{i-1}}{\epsilon^2 \eta_i} \frac{\max_{v \in V_i} \mathbb{E}[\hat{\Gamma}(v|S_{i-1})]}{\max_{S \subseteq V_i, |S|=b} \mathbb{E}[\hat{\Gamma}(S|S_{i-1})]} \right).
\end{aligned}$$

The first equality is derived by plugging in the definitions of k_i and k_{i-1} and extracting the common term θ_i/η_i . The inequality is obtained by amplifying the fraction term and offsetting the same terms. The third equality is derived by neglecting the deducted fraction. The fourth equality is held by ignoring the lower-order term. The fifth equality is derived by replacing θ_{i-1} with its value. The final equality can be obtained by the definition of T_{mRR} .

8.8 Proof of Theorem 4.8

The space complexity of GAAS is determined by the graph storage and the GmRR structure. The graph requires $O(m)$ space in sparse form. In round i , the auxiliary arrays such as *flag* and *offset* are expected to store one entry for each maintained mRR-set, and thus require $O(\theta_i)$ space. Let $|R|$ denote the size of a random mRR-set in round i , so storing all θ_i mRR-sets requires $O(\theta_i \mathbb{E}[|R|])$ space. Therefore, the overall expected space complexity is $O(m + \max_i \theta_i \mathbb{E}[|R|])$. To bound $\mathbb{E}[|R|]$, consider any node $v \in V_i$, and define an indicator variable X_v , where $X_v = 1$ if $v \in R$, and $X_v = 0$ otherwise. Then, $|R| = \sum_{v \in V_i} X_v$. By linearity of expectation, $\mathbb{E}[|R|] = \sum_{v \in V_i} \Pr[v \in R]$. By the definition of mRR-set, we have $\mathbb{E}[\hat{\Gamma}(v|S_{i-1})] = \eta_i \Pr[v \in R]$. Therefore,

$$\mathbb{E}[|R|] = \sum_{v \in V_i} \frac{\mathbb{E}[\hat{\Gamma}(v|S_{i-1})]}{\eta_i} \leq \frac{n_i \max_{v \in V_i} \mathbb{E}[\hat{\Gamma}(v|S_{i-1})]}{\eta_i}. \text{ Combining this with the bound on } \theta_i \text{ in Theorem 4.7, the final space complexity of GAAS is } O \left(m + \max_i \frac{b n_i \ln n_i}{\epsilon^2} \frac{\max_{v \in V_i} \mathbb{E}[\hat{\Gamma}(v|S_{i-1})]}{\max_{S \subseteq V_i, |S|=b} \mathbb{E}[\hat{\Gamma}(S|S_{i-1})]} \right).$$

9 Additional Results

Time breakdown and Effect of ParallelUpdate. We present the detailed time breakdown results under the LT model in Figure 16. As illustrated, in GAAS (marked with $\star$), the mRR-set update phase accounts for a substantial portion of the total running time, while GAAS_{gen} spends nearly all its time on mRR-set generation. Overall, GAAS consistently achieves significantly better efficiency than GAAS_{gen}, confirming the effectiveness of our proposed mRR-set update kernel.

mRR-set Memory Usage. Figure 17 presents memory usage of mRR-sets, which exhibits occasional drops and gradual increases between them, while the peak usage remains at a similar level across rounds. The reason is that, to be inefficient in practice, a method does not generate the theoretical maximum number of mRR-sets upfront in each round. Instead, the methods start with a small number and double the mRR-sets until a qualified set of seeds are selected for that round. Moreover, Figure 1 (dot-dash line, log-scale), e.g., Figure 1(a) for DBLP, shows that the actual number of mRR-sets used per round remains stable for several consecutive rounds and then drops significantly in some rounds.

Therefore, as stated in Section 4.6, in later rounds it is unnecessary to retain all previous mRR-sets, which may be excessive. Thus, we periodically clear and regenerate mRR-sets only in few selected rounds. Specifically, considering the actual number of mRR-sets needed in a round cannot be known in advance, we only clear and regenerate mRR-sets when the remaining influence threshold is halved from its value at the previous regeneration round. This discards excess mRR-sets and yields the observed drops in memory usage. Because regeneration is infrequent, its overhead is amortized by the savings from reuse and update in subsequent rounds. The memory usage increases between drops because, as explained in Section 4.4, the required number of roots in each mRR-set is monotonically non-decreasing over rounds, making a mRR-set larger and increasing memory usage. Moreover, if the previous mRR-sets are insufficient for this round, GAAS generates additional mRR-sets as

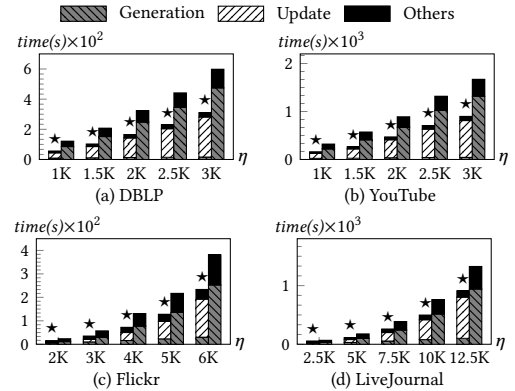


Figure 16: Runtime breakdown under the LT model; for each η , the left bar with $\star$ is GAAS and the right bar is GAAS_{gen}.

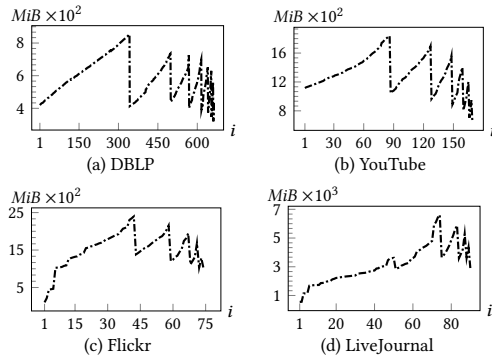


Figure 17: mRR-set Memory Usage across Rounds.

clarified in Section 4.3, also contributing to memory usage. The ASTI baseline exhibits a similar memory pattern because it always generates mRR-sets from scratch and also has the number of used mRR-sets dropped in certain rounds.

Given the similar peak memory usage across rounds and the complex pattern of increases and drops, we maintain the pre-allocated GmRR space without resizing it across rounds, since deciding when and by how much to resize it would introduce additional complexity and overhead.

References

- [1] 2023. INSIGHT COMPUTE COMMAND LINE INTERFACE. <https://docs.nvidia.com/nsight-compute/pdf/NsightComputeCli.pdf>.
- [2] 2026. Technical Report. https://github.com/gongyguo/GAAS/blob/main/GAAS_Technical_Report.pdf.
- [3] 360iResearch. 2026. Social Media Advertising Market – Global Forecast 2026-2032. (2026), 1–187.
- [4] Song Bian, Qintian Guo, Sibow Wang, and Jeffrey Xu Yu. 2020. Efficient Algorithms for Budgeted Influence Maximization on Massive Social Networks. *VLDB* 13, 9 (2020), 1498–1510.
- [5] Jiaping Cao, Le Xu, Man Lung Yiu, Jianbin Qin, and Bo Tang. 2025. GPH: An Efficient and Effective Perfect Hashing Scheme for GPU Architectures. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [6] Bogdan Cautis, Silviu Maniu, and Nikolaos Tziortziotis. 2019. Adaptive influence maximization. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3185–3186.
- [7] Wei Chen, Chi Wang, and Yajun Wang. 2010. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1029–1038.
- [8] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering* 5, 1 (1998), 46–55.
- [9] Gianlorenzo D’Angelo, Debashmita Poddar, and Cosimo Vinci. 2021. Improved Approximation Factor for Adaptive Influence Maximization via Simple Greedy Strategies. In *48th International Colloquium on Automata, Languages, and Programming*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 59–1.
- [10] Datareportal. 2025. Global social media statistics. <https://datareportal.com/social-media-users>.
- [11] Hossein Esfandiari, Amin Karbasi, and Vahab S. Mirrokni. 2021. Adaptivity in Adaptive Submodularity. In *COLT*, Vol. 134. 1823–1846.
- [12] Chen Feng, Xingguang Chen, Qintian Guo, Fangyuan Zhang, and Sibow Wang. 2024. Efficient Approximation Algorithms for Minimum Cost Seed Selection with Probabilistic Coverage Guarantee. *Proc. ACM Manag. Data* 2, 4 (2024), 1–26.
- [13] Daniel Golovin and Andreas Krause. 2011. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *J. Artif. Intell. Res.* 42 (2011), 427–486.
- [14] Daniel Golovin and Andreas Krause. 2017. Adaptive Submodularity: Theory and Applications in Active Learning and Stochastic Optimization. arXiv:1003.3967 [cs.LG] <https://arxiv.org/abs/1003.3967>
- [15] Amit Goyal, Francesco Bonchi, Laks VS Lakshmanan, and Suresh Venkatasubramanian. 2013. On minimizing budget and time in influence propagation over social networks. *Social Netw. Anal. Mining* 3 (2013), 179–192.
- [16] Qintian Guo, Chen Feng, Fangyuan Zhang, and Sibow Wang. 2023. Efficient Algorithm for Budgeted Adaptive Influence Maximization: An Incremental RR-set Update Approach. *Proc. ACM Manag. Data* 1, 3 (2023), 1–26.
- [17] Qintian Guo, Sibow Wang, Zhewei Wei, and Ming Chen. 2020. Influence maximization revisited: Efficient reverse reachable set generation with bound tightened. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2167–2181.
- [18] Kai Han, Yuntian He, Keke Huang, Xiaokui Xiao, Shaojie Tang, Jingxin Xu, and Liusheng Huang. 2019. Best bang for the buck: Cost-effective seed selection for online social networks. 32, 12 (2019), 2297–2309.
- [19] Kai Han, Keke Huang, Xiaokui Xiao, Jing Tang, Aixin Sun, and Xueyan Tang. 2018. Efficient algorithms for adaptive influence maximization. *VLDB* 11, 9 (2018), 1029–1040.
- [20] Wenjing Hong, Chao Qian, and Ke Tang. 2020. Efficient minimum cost seed selection with theoretical guarantees for competitive influence maximization. *IEEE transactions on cybernetics* 51, 12 (2020), 6091–6104.
- [21] Lin Hu, Yinnian Lin, Lei Zou, and M Tamer Özsu. 2025. A graph pattern mining framework for large graphs on GPU. *The VLDB Journal* 34, 1 (2025), 6.
- [22] Keke Huang, Jing Tang, Kai Han, Xiaokui Xiao, Wei Chen, Aixin Sun, Xueyan Tang, and Andrew Lim. 2020. Efficient approximation algorithms for adaptive influence maximization. *The VLDB Journal* 29, 6 (2020), 1385–1406.
- [23] Keke Huang, Sibow Wang, Glenn Bevilacqua, Xiaokui Xiao, and Laks VS Lakshmanan. 2017. Revisiting the stop-and-stare algorithms for influence maximization. (2017).
- [24] Yiqian Huang, Shiqi Zhang, Laks V. S. Lakshmanan, Wenqing Lin, Xiaokui Xiao, and Bo Tang. 2024. Efficient and Effective Algorithms for A Family of Influence Maximization Problems with A Matroid Constraint. *Proc. VLDB Endow.* 18, 2 (2024), 117–129.
- [25] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *KDD*. 137–146.
- [26] Andreas Krause and Carlos Guestrin. 2005. *A note on the budgeted maximization of submodular functions*. Carnegie Mellon University. Center for Automated Learning and Discovery.
- [27] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [28] Yinnian Lin, Lei Zou, and Xunbin Su. 2024. Towards Sufficient GPU-accelerated Dynamic Graph Management: Survey and Experiment. *Proc. VLDB Endow.* 18, 3 (2024), 599–612.
- [29] Hang Liu, H. Howie Huang, and Yang Hu. 2016. iBFS: Concurrent Breadth-First Search on GPUs. In *SIGMOD Conference*. ACM, 403–416.
- [30] Cheng Long and Raymond Chi-Wing Wong. 2011. Minimizing seed set for viral marketing. In *2011 IEEE 11th International Conference on Data Mining*. IEEE, 427–436.
- [31] Marco Minutoli, Maurizio Drocco, Mahantesh Halappanavar, Antonino Tumeo, and Ananth Kalyanaraman. 2020. cuRipples: Influence maximization on multi-GPU systems. In *Proceedings of the 34th ACM international conference on supercomputing*. 1–11.
- [32] Marco Minutoli, Mahantesh Halappanavar, Ananth Kalyanaraman, Arun Sathnur, Ryan McClure, and Jason McDermott. 2019. Fast and scalable implementations of influence maximization algorithms. In *2019 IEEE International Conference on Cluster Computing*. 1–12.
- [33] Hung T Nguyen, Tri P Nguyen, NhatHai Phan, and Thang N Dinh. 2017. Importance sketching of influence dynamics in billion-scale networks. In *2017 IEEE International Conference on Data Mining (ICDM)*. IEEE, 337–346.
- [34] Hung T Nguyen, My T Thai, and Thang N Dinh. 2016. Stop-and-stare: Optimal sampling algorithms for viral marketing in billion-scale networks. In *Proceedings of the 2016 international conference on management of data*. 695–710.
- [35] Binghui Peng and Wei Chen. 2019. Adaptive influence maximization with myopic feedback. *Advances in Neural Information Processing Systems* 32 (2019).
- [36] Linshan Qiu, Lu Chen, Hailiang Jie, Xiangyu Ke, Yunjun Gao, Yang Liu, and Zetao Zhang. 2024. GPU-Accelerated Batch-Dynamic Subgraph Matching. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3204–3216.
- [37] Soheil Shahrouz, Saber Salehkaleybar, and Matin Hashemi. 2021. gim: Gpu accelerated ris-based influence maximization algorithm. *IEEE Transactions on Parallel and Distributed Systems* 32, 10 (2021), 2386–2399.
- [38] Jieming Shi, Renchi Yang, Tianyuan Jin, Xiaokui Xiao, and Yin Yang. 2019. Real-time Top-k Personalized PageRank over Large Graphs on GPUs. *Proc. VLDB Endow.* 13, 1 (2019), 15–28.
- [39] Jing Tang, Keke Huang, Xiaokui Xiao, Laks VS Lakshmanan, Xueyan Tang, Aixin Sun, and Andrew Lim. 2019. Efficient approximation algorithms for adaptive seed minimization. In *SIGMOD*. 1096–1113.
- [40] Jing Tang, Xueyan Tang, Xiaokui Xiao, and Junsong Yuan. 2018. Online Processing Algorithms for Influence Maximization. In *SIGMOD*. 991–1005.
- [41] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence maximization in near-linear time: A martingale approach. In *SIGMOD*. 1539–1554.
- [42] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: near-optimal time complexity meets practical efficiency. In *SIGMOD*. 75–86.

- [43] Guangmo Tong, Weili Wu, Shaojie Tang, and Ding-Zhu Du. 2016. Adaptive influence maximization in dynamic social networks. *IEEE/ACM Transactions on Networking* 25, 1 (2016), 112–125.
- [44] Sharan Vaswani and Laks VS Lakshmanan. 2016. Adaptive influence maximization in social networks: Why commit when you can adapt? *arXiv preprint arXiv:1604.08171* (2016).
- [45] Jinghao Wang, Yanping Wu, Xiaoyang Wang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2024. Efficient Influence Minimization via Node Blocking. *Proc. VLDB Endow.* 17, 10 (2024), 2501–2513.
- [46] Letong Wang, Xiangyun Ding, Yan Gu, and Yihan Sun. 2023. Fast and Space-Efficient Parallel Algorithms for Influence Maximization. *VLDB* 17, 3 (2023), 400–413.
- [47] Yexin Wang, Zhi Yang, Junqi Liu, Wentao Zhang, and Bin Cui. 2023. Scapin: Scalable Graph Structure Perturbation by Augmented Influence Maximization. *Proc. ACM Manag. Data* 1, 2 (2023), 146:1–146:21.
- [48] Yifei Xia, Feng Zhang, Qingyu Xu, Mingde Zhang, Zhiming Yao, Lv Lu, Xiaoyong Du, Dong Deng, Bingsheng He, and Siqi Ma. 2024. GPU-based butterfly counting. *The VLDB Journal* 33, 5 (2024), 1543–1567.
- [49] Qingyu Xu, Feng Zhang, Zhiming Yao, Lv Lu, Xiaoyong Du, Dong Deng, and Bingsheng He. 2022. Efficient Load-Balanced Butterfly Counting on GPU. *Proc. VLDB Endow.* 15, 11 (2022), 2450–2462.
- [50] Yichao Yuan, Haojie Ye, Sanketh Vedula, Wynn Kaza, and Nishil Talati. 2023. Everest: GPU-Accelerated System For Mining Temporal Motifs. *Proc. VLDB Endow.* 17, 2 (2023), 162–174.
- [51] Peng Zhang, Wei Chen, Xiaoming Sun, Yajun Wang, and Jialin Zhang. 2014. Minimizing seed set selection with probabilistic coverage guarantee in a social network. In *KDD*. 1306–1315.
- [52] Yapu Zhang, Shengminjie Chen, Wenqing Xu, and Zhenning Zhang. 2022. Adaptive influence maximization under fixed observation time-step. *Theoretical Computer Science* 928 (2022), 104–114.